



รายงาน

การศึกษา Automated Test Case Generation Algorithms และ AI-Assisting Tools
กรณีศึกษา Whole Test Suite Generation (WTS), Coverage-guided Greybox
Fuzzing, Claude AI sonet 5-5 และ Gemini 3.8 flash สำหรับการสร้างชุดทดสอบ
ใน Java Projects ของ Defects4J

จัดทำโดย

นายพชรพงศ์ สาหลอน	รหัสนักศึกษา 673380414-7
นายนันทกร แสงวงจิตร	รหัสนักศึกษา 673380085-0
นายศุภวัฒน์ แสนเรียน	รหัสนักศึกษา 673380604-2

เสนอ

ผู้ช่วยศาสตราจารย์ ดร.ชิตสุธา สุ่มเล็ก
วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

รายงานนี้เป็นส่วนหนึ่งของรายวิชา CP353201 การประกันคุณภาพซอฟต์แวร์
Software Quality Assurance วิทยาลัยการคอมพิวเตอร์ สาขาวิทยาการคอมพิวเตอร์
มหาวิทยาลัยขอนแก่น ภาคการศึกษาที่ 1 ปีการศึกษา 2569

บทคัดย่อ

การสร้างชุดทดสอบเป็นกระบวนการสำคัญในการประกันคุณภาพซอฟต์แวร์ เนื่องจากช่วยตรวจสอบความถูกต้องของโปรแกรมและค้นหาข้อบกพร่องที่อาจเกิดขึ้น อย่างไรก็ตาม การสร้างชุดทดสอบด้วยมือสำหรับโครงการขนาดใหญ่อาจใช้เวลาและทรัพยากรจำนวนมาก โครงการนี้จึงมุ่งศึกษาแนวทางการสร้างชุดทดสอบ 4 แนวทาง ได้แก่ Whole Test Suite Generation (WTS), Coverage-guided Greybox Fuzzing, Claude AI Sonnet 5.5 และ Gemini 3.8 Flash โดยใช้ Defects4J เป็นชุดข้อมูลสำหรับการทดลองกับโครงการภาษา Java

การทดลองแบ่งออกเป็น Automated Test Case Generation Algorithms และ AI-Assisting Tools โดย WTS ใช้ EvoSuite ในการสร้าง JUnit Test Suite และ Coverage-guided Greybox Fuzzing ใช้ Coverage Feedback ในการสร้างและคัดเลือกข้อมูลทดสอบ ส่วน Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ใช้ความสามารถของ Generative AI ผ่าน KKKU IntelSphere API เพื่อสร้างชุดทดสอบระดับ Unit Testing จากข้อมูลของ Bug และ Source Code โดยมีการสร้าง Prompt ตามข้อมูลของแต่ละ Bug และจัดเก็บผลลัพธ์เพื่อใช้ในการประเมิน

การประเมินผลพิจารณาจากความสามารถในการ Compile และ Execute, Test Coverage และ Fault Detection Rate (FDR) ผลการทดลองที่สามารถวัดผลได้แสดงให้เห็นว่า AI-Assisting Tools สามารถสร้างชุดทดสอบที่ตรวจจับข้อบกพร่องจริงได้ และสามารถนำไปใช้ร่วมกับกระบวนการประเมิน Test Coverage ได้ อย่างไรก็ตาม ผลการทดลองของแต่ละแนวทางมีขอบเขตและเงื่อนไขที่แตกต่างกัน รวมถึงข้อจำกัดด้าน API Quota และระยะเวลาในการทดลอง

ดังนั้น การเปรียบเทียบประสิทธิภาพจึงพิจารณาตามชุดข้อมูลและเงื่อนไขที่สามารถเปรียบเทียบกันได้ โดยไม่นำข้อมูลที่ยังไม่ได้ทดลองมาประมาณหรือสร้างขึ้น

คำสำคัญ การสร้างชุดทดสอบอัตโนมัติ, Automated Test Case Generation, AI-Assisting Tools, Whole Test Suite Generation, Coverage-guided Greybox Fuzzing, EvoSuite, Defects4J, Test Coverage, Fault Detection Rate

คำนำ

การทดสอบซอฟต์แวร์เป็นส่วนสำคัญของกระบวนการพัฒนาซอฟต์แวร์ เนื่องจากช่วยตรวจสอบความถูกต้องของการทำงานและค้นหาข้อบกพร่องที่อาจเกิดขึ้นก่อนนำซอฟต์แวร์ไปใช้งานจริง อย่างไรก็ตาม เมื่อซอฟต์แวร์มีขนาดใหญ่และมีความซับซ้อนมากขึ้น การสร้างชุดทดสอบด้วยมืออาจต้องใช้เวลาและทรัพยากรจำนวนมาก จึงเกิดการพัฒนาระบบการสร้างชุดทดสอบอัตโนมัติที่สามารถช่วยลดภาระของผู้ทดสอบและเพิ่มความครอบคลุมในการทดสอบ

ปัจจุบันมีแนวทางการสร้างชุดทดสอบหลายรูปแบบ ทั้งแนวทางที่อาศัยกระบวนการค้นหาและข้อมูลจากการทำงานของโปรแกรม เช่น Whole Test Suite Generation และ Coverage-guided Greybox Fuzzing รวมถึงแนวทางที่นำความสามารถของ Generative AI มาใช้ในการวิเคราะห์ Source Code และสร้าง Test Code จากข้อมูลและคำสั่งภาษามนุษย์ เทคโนโลยีดังกล่าวมีความสามารถและข้อจำกัดที่แตกต่างกัน จึงมีความน่าสนใจในการศึกษาและเปรียบเทียบประสิทธิภาพภายใต้สภาพแวดล้อมการทดลองที่เหมาะสม

รายงานนี้เป็นส่วนหนึ่งของรายวิชา CP353201 การประกันคุณภาพซอฟต์แวร์ (Software Quality Assurance) โดยจัดทำขึ้นเพื่อศึกษาแนวทางการสร้างชุดทดสอบ 4 แนวทาง ได้แก่ Whole Test Suite Generation (WTS), Coverage-guided Greybox Fuzzing, Claude AI Sonnet 5.5 และ Gemini 3.8 Flash โดยใช้ Defects4J เป็นชุดข้อมูลสำหรับการทดลอง และพิจารณาผลการสร้างชุดทดสอบจาก Test Coverage และความสามารถในการตรวจจับข้อบกพร่อง รวมถึงข้อจำกัดที่เกิดขึ้นระหว่างกระบวนการทดลอง เนื้อหาในรายงานฉบับนี้ประกอบด้วยแนวคิดและทฤษฎีที่เกี่ยวข้อง วิธีดำเนินการทดลอง ผลการทดลอง การวิเคราะห์และอภิปรายผล ข้อจำกัดของการศึกษา ตลอดจนข้อสรุปและข้อเสนอแนะ เพื่อเป็นแนวทางสำหรับการศึกษการสร้างชุดทดสอบอัตโนมัติและการประยุกต์ใช้ AI-Assisting Tools ในการทดสอบซอฟต์แวร์ต่อไป

ผู้จัดทำหวังว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อการศึกษาและทำความเข้าใจแนวทางการสร้างชุดทดสอบอัตโนมัติ ตลอดจนการประยุกต์ใช้ AI-Assisting Tools ในงานด้านการประกันคุณภาพซอฟต์แวร์

คณะผู้จัดทำ

วันที่ 4 ตุลาคม 2569

สารบัญ

	หน้า
บทคัดย่อ	ก
คำนำ	ข
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญของโครงการ	
1.2 วัตถุประสงค์ของโครงการ	2
1.3 ขอบเขตของโครงการ	
บทที่ 2 แนวคิด ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง	4
2.1 การสร้างกรณีทดสอบอัตโนมัติ	
2.3 การออกแบบ Prompt สำหรับการสร้าง Unit Test	7
2.4 ชุดข้อมูล Defects4J	8
2.5 ตัวชี้วัดในการประเมินการสร้างกรณีทดสอบ	9
บทที่ 3 วิธีดำเนินการทดลอง	10
3.1 ภาพรวมการดำเนินการทดลอง	
3.2 การเตรียมข้อมูลการทดลอง	
3.3 การเตรียมอัลกอริทึมการสร้างกรณีทดสอบ	
3.4 การเตรียม Generative AI	12
3.5 การสร้างกรณีทดสอบ	13
3.6 การดำเนินการทดสอบ	
3.7 การวัด Test Coverage	14
3.8 การวัดความสามารถในการตรวจจับข้อบกพร่อง	
3.9 การเปรียบเทียบผลการทดลอง.	15
บทที่ 4 ผลการทดลอง	16
4.1 ผลการทดลอง Whole Test Suite Generation	
4.2 ผลการทดลอง Coverage-guided Greybox Fuzzing	
4.3 ผลการทดลอง Claude Sonnet 5.5	17
4.4 ผลการทดลอง Gemini 3.8 Flash	18
4.5 ผลการวัด Test Coverage	20
4.6 ผลการวัด Fault Detection	
บทที่ 5 วิเคราะห์และอภิปรายผล	22
5.1 การเปรียบเทียบ Whole Test Suite Generation	

และ Coverage-guided Greybox Fuzzing

5.2 การเปรียบเทียบ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash	
5.3 การเปรียบเทียบทั้ง 4 แนวทาง	24
5.4 ความสัมพันธ์ระหว่าง Test Coverage และ Fault Detection	25
5.5 การวิเคราะห์รูปแบบความล้มเหลว	28
บทที่ 6 ข้อจำกัด ปัญหา และความสามารถในการทำซ้ำการทดลอง	29
6.1 ข้อจำกัดของการทดลอง	
6.2 ปัญหาและอุปสรรคในการดำเนินการทดลอง	30
6.3 ความสามารถในการทำซ้ำการทดลอง	32
บทที่ 7 สรุปผลและข้อเสนอแนะ	33
7.1 สรุปผลการศึกษา	
7.2 สรุปผลการเปรียบเทียบทั้ง 4 แนวทาง	34
7.3 ข้อจำกัดที่มีผลต่อการสรุปผล	
7.4 ข้อเสนอแนะสำหรับการดำเนินงานต่อไป	35
เอกสารอ้างอิง	36

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญของโครงการ

การทดสอบซอฟต์แวร์เป็นขั้นตอนสำคัญของกระบวนการพัฒนาซอฟต์แวร์ เนื่องจากซอฟต์แวร์อาจยังมีข้อบกพร่องที่ซ่อนอยู่ในเส้นทางการทำงานบางรูปแบบ การทดสอบจึงมีหน้าที่ตรวจสอบพฤติกรรมและผลลัพธ์ของระบบ เพื่อช่วยลดความเสี่ยงของข้อผิดพลาดเมื่อซอฟต์แวร์ถูกนำไปใช้งานจริง

การสร้างกรณีทดสอบเป็นหนึ่งในขั้นตอนที่ใช้ทรัพยากรจำนวนมาก โดยเฉพาะโครงการที่มี Source Code จำนวนมากหรือมีเส้นทางการทำงานที่ซับซ้อน การสร้าง Test ด้วยมืออาจไม่สามารถครอบคลุมเส้นทางการทำงานที่เป็นไปได้ทั้งหมด จึงมีการพัฒนาแนวทางการสร้างกรณีทดสอบอัตโนมัติ เช่น Search-Based Software Testing และ Fuzzing เพื่อช่วยสร้างและปรับปรุงชุดทดสอบโดยอาศัยข้อมูลจากการทำงานของโปรแกรม

Whole Test Suite Generation (WTS) เป็นแนวทางหนึ่งของ Search-Based Software Testing ที่พิจารณา Test Suite เป็นชุดโดยรวม และใช้กระบวนการค้นหาเพื่อปรับปรุงชุดทดสอบให้บรรลุเป้าหมายด้าน Coverage ที่กำหนด โดย EvoSuite เป็นเครื่องมือที่นำแนวคิดดังกล่าวมาใช้กับภาษา Java ขณะที่ Coverage-guided Greybox Fuzzing ใช้ข้อมูล Coverage เป็น Feedback เพื่อคัดเลือก Input ที่สามารถสำรวจเส้นทางการทำงานใหม่และนำไปสร้าง Input เพิ่มเติม

นอกจากแนวทางดังกล่าวแล้ว ความก้าวหน้าของ Generative AI ทำให้เกิด AI-Assisting Tools ที่สามารถวิเคราะห์ Source Code และข้อมูลเกี่ยวกับ Bug เพื่อสร้าง Test Code ได้ การศึกษานี้จึงเลือกใช้ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash เป็น AI-Assisting Tools สำหรับสร้างชุดทดสอบระดับ Unit Testing อย่างไรก็ตาม Test Code ที่สร้างขึ้นอาจ Compile ไม่ผ่าน ไม่มี Assertion ที่เหมาะสม หรือไม่สามารถตรวจจับข้อบกพร่องได้ จึงจำเป็นต้องมีการประเมินผลอย่างเป็นระบบ

โครงการนี้เลือกใช้ Defects4J ซึ่งเป็นชุดข้อมูลข้อบกพร่องจริงจากโครงการภาษา Java และมีทั้ง Buggy Version และ Fixed Version ทำให้สามารถตรวจสอบความสามารถของชุดทดสอบในการตรวจจับข้อบกพร่องได้

ดังนั้น การศึกษานี้จึงมุ่งศึกษาและเปรียบเทียบแนวทางการสร้างชุดทดสอบ 4 แนวทาง ได้แก่ Whole Test Suite Generation (WTS), Coverage-guided Greybox Fuzzing, Claude AI Sonnet 5.5 และ Gemini 3.8 Flash โดยพิจารณา Test Coverage, Fault Detection Rate, จำนวน Test และระยะเวลาในการสร้างชุดทดสอบ รวมถึงข้อจำกัดที่เกิดขึ้นระหว่างการทดลอง

1.2 วัตถุประสงค์ของโครงการ

1. เพื่อศึกษาแนวคิดและกระบวนการทำงานของ Automated Test Case Generation Algorithms และ AI-Assisting Tools สำหรับการสร้างชุดทดสอบระดับ Unit Testing
2. เพื่อศึกษาและประยุกต์ใช้ Whole Test Suite Generation (WTS) ผ่านเครื่องมือ EvoSuite สำหรับโครงการภาษา Java ใน Defects4J
3. เพื่อศึกษาและออกแบบ Coverage-guided Greybox Fuzzing สำหรับการสร้างข้อมูลทดสอบและชุดทดสอบ
4. เพื่อศึกษาและประยุกต์ใช้ AI-Assisting Tools ได้แก่ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ในการสร้างชุดทดสอบระดับ Unit Testing
5. เพื่อออกแบบ Prompt และกระบวนการสร้างชุดทดสอบที่สามารถดำเนินการอย่างเป็นระบบและสามารถทำซ้ำได้
6. เพื่อประเมินความสามารถของชุดทดสอบในการตรวจจับข้อบกพร่องจริงใน Defects4J และวัด Test Coverage ของชุดทดสอบที่สร้างขึ้น
7. เพื่อวิเคราะห์ปัจจัยที่ส่งผลต่อความสามารถในการสร้างชุดทดสอบและการตรวจจับข้อบกพร่อง
8. เพื่อเปรียบเทียบจุดเด่น ข้อจำกัด ประสิทธิภาพ และระยะเวลาในการสร้างชุดทดสอบของทั้ง 4 แนวทาง ได้แก่ Whole Test Suite Generation, Coverage-guided Greybox Fuzzing, Claude AI Sonnet 5.5 และ Gemini 3.8 Flash

1.3 ขอบเขตของโครงการ

1.3.1 ขอบเขตด้านชุดข้อมูล Defects4J การทดลองใช้ชุดข้อมูล Defects4J เวอร์ชัน 3.0.1 ซึ่งประกอบด้วยโครงการภาษา Java และข้อบกพร่องจริงจำนวน 854 Active Bugs จาก 17 โครงการ ได้แก่ Lang, Math, Chart, Closure, Time, Cli, Codec, Collections, Compress, Csv, Gson, JacksonCore, JacksonDatabind, JacksonXml, Jsoup, JXPath และ Mockito การทดสอบแต่ละ Bug พิจารณา Buggy Version และ Fixed Version เพื่อประเมินความสามารถของชุดทดสอบในการตรวจจับข้อบกพร่อง

1.3.2 ขอบเขตด้าน Automated Test Case Generation Algorithm การศึกษาด้าน Automated Test Case Generation Algorithms แบ่งออกเป็น 2 แนวทาง ได้แก่ Whole Test Suite Generation และ Coverage-guided Greybox Fuzzing

1.3.2.1 Whole Test Suite Generation ใช้ EvoSuite เป็นเครื่องมือสำหรับสร้าง JUnit Test Suite โดยกำหนดเป้าหมายการค้นหา ได้แก่ Branch, Line, Exception และ Weak Mutation กำหนดงบประมาณ 120 วินาทีต่อ Bug ใช้ Random Seed เท่ากับ 1 และใช้คลาสที่ได้รับการแก้ไขของ Bug เป็น Target Class

1.3.2.2 Coverage-guided Greybox Fuzzing คือการศึกษาการสร้างข้อมูลทดสอบโดยใช้ Coverage Feedback ประกอบด้วยกระบวนการ Seed Selection, Mutation, Execution, Coverage Feedback และ Test Export โดยออกแบบให้สามารถประยุกต์ใช้กับการทดสอบระดับ Method หรือ Class ได้ การทดลองกำหนด Configuration ประกอบด้วย Budget 30, 60 และ 120 วินาทีต่อคลาส Seed Corpus หลายรูปแบบ และ Mutation Operators หลายระดับ โดยกำหนดการทำซ้ำ 5 รอบต่อ Configuration เพื่อใช้ในการวิเคราะห์ผลการทดลอง

1.3.3 ขอบเขตด้าน AI-Assisting Tools การทดลองด้าน AI-Assisting Tools ใช้ 2 เครื่องมือ ได้แก่

- 1 Claude AI Sonnet 5.5
- 2 Gemini 3.8 Flash

ทั้งสองเครื่องมือใช้งานผ่าน KKU IntelSphere API เพื่อสร้างชุดทดสอบระดับ Unit Testing สำหรับโครงการภาษา Java ใน Defects4J โดยใช้โปรแกรม Java ผ่าน Maven และ Java 17 สำหรับการเรียก API ส่วนการดำเนินการกับ Defects4J และการวัด Coverage ใช้ Java 11 และกำหนด max_tokens = 4096 สำหรับการเรียกใช้ API

สำหรับ Gemini มีการเพิ่ม EFFICIENCY NOTE ใน Prompt เพื่อควบคุมการใช้ Thinking Token หลังจากการทดลองเบื้องต้นพบว่าการใช้ Thinking Token อาจทำให้ Response ไม่สมบูรณ์ภายใต้ข้อจำกัดของ max_tokens โดยเงื่อนไขดังกล่าวถูกกำหนดเป็นส่วนหนึ่งของการทดลองและนำมาพิจารณาในการวิเคราะห์ผล

บทที่ 2

แนวคิด ทฤษฎี และเทคโนโลยีที่เกี่ยวข้อง

2.1 การสร้างกรณีทดสอบอัตโนมัติ

2.1.1 แนวคิดการสร้างกรณีทดสอบอัตโนมัติ

Automated Test Case Generation คือแนวทางที่ใช้กระบวนการอัตโนมัติในการสร้างกรณีทดสอบ แทนการออกแบบ Test Case ด้วยมือทั้งหมด โดยมีวิธีการและเป้าหมายที่แตกต่างกัน เช่น การเพิ่ม Code Coverage การค้นหาข้อบกพร่อง การสร้าง Input ที่มีลักษณะผิดปกติ และการตรวจสอบพฤติกรรมของโปรแกรมตามเงื่อนไขที่กำหนด

การประเมินประสิทธิภาพของการสร้างกรณีทดสอบอัตโนมัติไม่ควรพิจารณาจาก Coverage เพียงอย่างเดียว เนื่องจาก Coverage ที่สูงไม่ได้หมายความว่าชุดทดสอบจะสามารถตรวจจับข้อบกพร่องได้สูงเสมอไป ชุดทดสอบอาจสามารถเข้าถึง Statement หรือ Branch ของโปรแกรมได้ แต่ไม่มี Assertion ที่เหมาะสมในการตรวจสอบผลลัพธ์ ดังนั้น การประเมินจึงควรใช้ตัวชี้วัดหลายด้านร่วมกัน เช่น Test Coverage, Fault Detection Rate, จำนวน Test และระยะเวลาในการสร้างชุดทดสอบ เพื่อให้สามารถประเมินคุณภาพและประสิทธิภาพของชุดทดสอบได้อย่างเหมาะสม

2.1.2 Whole Test Suite Generation Whole Test Suite Generation (WTS) เป็นแนวทางหนึ่งของ Search-Based Software Testing ที่พิจารณา Test Suite ทั้งชุดเป็น Candidate Solution และใช้กระบวนการค้นหาเพื่อปรับปรุงคุณภาพของชุดทดสอบ โดยอาศัยค่า Fitness หรือ Coverage เป็นข้อมูลสำหรับประเมินชุดทดสอบ

กระบวนการทั่วไปของ WTS ประกอบด้วยการสร้าง Candidate Test Suite การนำชุดทดสอบไปทำงาน การประเมิน Fitness หรือ Coverage การคัดเลือกชุดทดสอบที่มีคุณภาพ และการสร้างหรือปรับปรุง Test เพิ่มเติม กระบวนการดังกล่าวจะดำเนินซ้ำจนกว่าจะบรรลุเงื่อนไขที่กำหนด จากนั้นจึงลด Test ที่ไม่จำเป็นเพื่อให้ได้ Final Test Suite ที่เหมาะสม

แนวทางดังกล่าวแตกต่างจากการสร้าง Test Case แยกตามเป้าหมาย เนื่องจาก Test หลายรายการภายใน Test Suite สามารถทำงานร่วมกันเพื่อค้นหาเส้นทางการทำงานและเพิ่ม Coverage ของโปรแกรมได้

2.1.3 เครื่องมือ EvoSuite EvoSuite เป็นเครื่องมือสำหรับการสร้าง JUnit Test Suite สำหรับโครงการภาษา Java โดยใช้แนวคิด Evolutionary Search และ Whole Test Suite Generation ในการค้นหาและปรับปรุงชุดทดสอบให้สามารถบรรลุเป้าหมายด้าน Coverage ที่กำหนด

ในการศึกษา EvoSuite ถูกนำมาใช้เป็นเครื่องมือสำหรับการทดลอง Whole Test Suite Generation โดยใช้ evosuite-current.jar ที่มาพร้อมกับสภาพแวดล้อมของ Defects4J เพื่อให้สามารถทำงานร่วมกับ Classpath ของโครงการที่ใช้ในการทดลองได้อย่างเหมาะสม

Configuration หลักที่ใช้ในการทดลองประกอบด้วย

ตารางที่ 1 ค่าการกำหนดสำหรับการทดลอง Whole Test Suite Generation ด้วย EvoSuite

Parameter	ค่า
Defects4J	3.0.1
Total Budget	120 วินาที
Criterion	Branch, Line, Exception, Weak Mutation
Seed	1
Stopping Condition	MaxTime
Target	classes.modified
Project Classpath	defects4j export -p cp.compile

กระบวนการทำงานถูกจัดทำเป็น Shell Script เพื่อให้สามารถ Checkout Bug, Compile, Export Classpath, เรียก EvoSuite, บันทึกผล และทำความสะอาด Checkout ได้อย่างเป็นระบบ

2.1.4 Coverage-guided Greybox Fuzzing Coverage-guided Greybox Fuzzing เป็นแนวทางการสร้างข้อมูลทดสอบที่ใช้ Feedback จากการทำงานของโปรแกรมเพื่อควบคุมกระบวนการสร้าง Input โดยเริ่มจาก Seed Corpus แล้วนำ Seed ไปผ่านกระบวนการ Mutation เพื่อสร้าง Input ใหม่ จากนั้นนำ Input ที่ได้ไป Execute และตรวจสอบ Coverage หาก Input ใหม่สามารถทำให้เกิดเส้นทางการทำงานหรือ Coverage ที่ไม่เคยพบมาก่อน Input ดังกล่าวจะถูกเก็บไว้ใน Corpus และสามารถนำไปใช้เป็น Seed สำหรับการสร้าง Input ในรอบถัดไป

กระบวนการหลักของ Coverage-guided Greybox Fuzzing ประกอบด้วย

1. การเตรียม Seed Corpus
2. การเลือก Seed
3. การสร้าง Input ใหม่ด้วย Mutation
4. การนำ Input ไป Execute
5. การตรวจสอบ Coverage Feedback
6. การตรวจสอบ Crash หรือ Exception
7. การทำซ้ำจนถึงเวลาที่กำหนด
8. การนำข้อมูลทดสอบที่ได้ไปสร้างเป็น Unit Test

Mutation Operator ที่สามารถนำมาใช้ในการทดลอง ได้แก่ Bit Flip, การเพิ่มหรือลดค่า, Boundary Value, Splice และ Dictionary-based Mutation โดยแต่ละรูปแบบมีลักษณะการเปลี่ยนแปลง Input ที่แตกต่างกัน และอาจส่งผลต่อความสามารถในการค้นหาเส้นทางการทำงานของโปรแกรม

เนื่องจากโครงงานนี้มุ่งเน้นการสร้างชุดทดสอบระดับ Unit Testing จึงจำเป็นต้องมี Fuzz Target หรือ Harness สำหรับกำหนด Method หรือ Class เป้าหมาย โดย Input ที่สร้างจาก Fuzzer จะถูกแปลงให้อยู่ในรูปแบบข้อมูลที่ Method สามารถรับได้ เช่น int, String, List, Object หรือค่าพารามิเตอร์ประเภทอื่น ๆ ในกรณีที่ Method ต้องใช้ Object ที่มีโครงสร้างซับซ้อน อาจจำเป็นต้องสร้าง Instance ผ่าน Constructor และเรียกใช้ Method ตามลำดับที่เหมาะสมเพื่อให้เกิดสถานะภายในที่แตกต่างกัน

สำหรับการตรวจสอบผลลัพธ์ของชุดทดสอบ เนื่องจาก Fuzzer ไม่สามารถทราบผลลัพธ์ที่ถูกต้องของทุก Input ได้โดยตรง โครงงานจึงกำหนดแนวทางให้ใช้ Fixed Version ของ Defects4J เป็น Regression Oracle โดยผลลัพธ์จาก Fixed Version สามารถใช้เป็น Expected Result สำหรับเปรียบเทียบกับผลลัพธ์จาก Buggy Version เพื่อประเมินความสามารถของชุดทดสอบในการตรวจจับข้อบกพร่องได้

2.2 AI-Assisting Tools สำหรับการสร้างชุดทดสอบ

2.2.1 แนวคิด Generative AI Generative AI เป็นเทคโนโลยีปัญญาประดิษฐ์ที่สามารถสร้างเนื้อหาใหม่จากข้อมูลและคำสั่งที่ได้รับ โดยสามารถนำมาประยุกต์ใช้ในการสร้าง Source Code และ Test Code จากคำอธิบายภาษาธรรมชาติได้

สำหรับการสร้างชุดทดสอบ Generative AI สามารถวิเคราะห์ข้อมูลที่เกี่ยวข้องกับโปรแกรม เช่น Source Code, API, Method และข้อมูลเกี่ยวกับ Bug แล้วนำข้อมูลดังกล่าวมาใช้ในการสร้าง Test Code ตามรูปแบบของ Test Framework เช่น JUnit การนำ Generative AI มาใช้ในลักษณะดังกล่าวจึงสามารถสนับสนุนกระบวนการสร้างชุดทดสอบระดับ Unit Testing ได้

2.2.2 Claude AI Sonnet 5.5 Claude AI Sonnet 5.5 เป็น AI-Assisting Tool ที่สามารถประมวลผลข้อมูลจาก Prompt และสร้าง Source Code หรือ Test Code ตามบริบทที่ได้รับ ในการศึกษาครั้งนี้ นำ Claude AI Sonnet 5.5 มาใช้สำหรับสร้างชุดทดสอบระดับ Unit Testing สำหรับโครงการภาษา Java ใน Defects4J โดยใช้ข้อมูลของ Bug, Source Code และบริบทที่เกี่ยวข้องเป็นข้อมูลประกอบการสร้าง Test Code

2.2.3 Gemini 3.8 Flash Gemini 3.8 Flash เป็น AI-Assisting Tool ที่สามารถวิเคราะห์ข้อมูลจาก Prompt และสร้าง Source Code หรือ Test Code ตามบริบทที่กำหนด ในการศึกษาครั้งนี้ นำ Gemini 3.8 Flash มาใช้สำหรับสร้างชุดทดสอบระดับ Unit Testing สำหรับโครงการภาษา Java ใน Defects4J โดยใช้ข้อมูลของ Bug, Source Code และบริบทที่เกี่ยวข้องเป็นข้อมูลประกอบการสร้าง Test Code

การใช้งาน Gemini ในการทดลองกำหนดข้อจำกัดด้านจำนวน Token ผ่านค่า max_tokens และมีการปรับ Prompt เพื่อควบคุมการใช้ Thinking Token ซึ่งรายละเอียดเกี่ยวกับรูปแบบ Prompt และเงื่อนไขการเรียกใช้ API จะอธิบายในบทที่ 3

2.3 การออกแบบ Prompt สำหรับการสร้าง Unit Test

การออกแบบ Prompt ในการทดลองมีวัตถุประสงค์เพื่อให้ AI-Assisting Tools สามารถสร้างชุดทดสอบระดับ Unit Test ที่เหมาะสมกับข้อบกพร่องแต่ละรายการใน Defects4J โดย Prompt ของแต่ละ Bug จะถูกสร้างจากข้อมูลของ Bug และโครงสร้างของโครงการนั้น ๆ แทนการใช้ Prompt แบบคงที่กับทุก Bug

เพื่อให้กระบวนการสร้าง Prompt มีความเป็นระบบและสามารถทำซ้ำได้ จึงพัฒนา PromptGenerator สำหรับสร้าง Prompt ของแต่ละ Bug โดยอัตโนมัติ และจัดเก็บ Prompt ที่ใช้จริงไว้ใน Repository เพื่อให้สามารถตรวจสอบข้อมูลที่ส่งให้ AI-Assisting Tools ได้

2.3.1 องค์ประกอบของ Prompt

Prompt ที่ใช้ในการทดลองประกอบด้วยข้อมูลสำคัญของ Bug ได้แก่

1. ข้อมูลของ Bug จาก Defects4J
2. ข้อมูลของ Trigger Test ที่เกี่ยวข้องกับ Bug
3. Source Code ที่เกี่ยวข้องกับ Bug
4. Diff ของ Source Code ระหว่าง Buggy Version และ Fixed Version
5. ข้อมูลของ Class หรือ Method ที่เกี่ยวข้องกับข้อบกพร่อง
6. ข้อกำหนดให้สร้างชุดทดสอบระดับ Unit Test
7. ข้อกำหนดให้ Test Code สามารถ Compile และ Execute ได้ในสภาพแวดล้อมของ Defects4J

Trigger Test ใช้เพื่อแสดงรูปแบบการเรียกใช้โปรแกรมและรูปแบบ Assertion ที่เกี่ยวข้องกับโครงการ ส่วน Source Code Diff ใช้เพื่อแสดงตำแหน่งและลักษณะของการเปลี่ยนแปลงที่เกี่ยวข้องกับข้อบกพร่อง ข้อมูลดังกล่าวช่วยให้ AI-Assisting Tools สามารถสร้าง Test Code โดยอาศัยบริบทของ Bug แต่ละรายการ

Prompt ของแต่ละ Bug ถูกจัดเก็บในรูปแบบ

```
Prompt/<Project>/<Project>-<N>/prompt.txt
```

การจัดเก็บในลักษณะดังกล่าวช่วยให้สามารถตรวจสอบย้อนหลังได้ว่าแต่ละ Bug ถูกนำข้อมูลใดมาใช้ในการสร้าง Prompt และช่วยสนับสนุนการทำซ้ำการทดลอง

2.3.2 การควบคุมความแตกต่างระหว่าง Claude และ Gemini

ในการทดลองใช้ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ผ่าน KKU IntelSphere API โดยกำหนด max_tokens = 4096 สำหรับการเรียกใช้ทั้งสองเครื่องมือ เนื่องจาก Gemini มีการใช้ Thinking Token ซึ่งอยู่ภายใต้ข้อจำกัดของ max_tokens จึงมีการเพิ่มข้อความ EFFICIENCY NOTE ใน Prompt ของ Gemini เพื่อควบคุมการใช้ Token และลดโอกาสที่ Response จะสิ้นสุดก่อนสร้าง Test Code ครบถ้วน โดยไม่ได้เพิ่มข้อความดังกล่าวใน Prompt ของ Claude

=== EFFICIENCY NOTE ===

Do not deliberate at length. Keep any internal reasoning very brief and write the final test code directly.

ความแตกต่างของ Prompt ดังกล่าวถือเป็นเงื่อนไขหนึ่งของการทดลอง และถูกนำมาพิจารณาเมื่อวิเคราะห์และอภิปรายผลการทดลองของ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash

2.3.3 การจัดเก็บ Prompt และ Response

Prompt ที่ใช้จริง รวมถึง Response ที่ได้รับจาก API ถูกจัดเก็บแยกตาม Project และ Bug เพื่อให้สามารถตรวจสอบกระบวนการทดลองย้อนหลังได้ Response ดิบจาก API ถูกเก็บไว้ใน

Result/<Project>/<Project>-<N>/raw_response.txt

ส่วน Test Code ที่ดึงออกจาก Response ถูกเก็บไว้ใน

TestCode/<Project>/<Project>-<N>/

ระบบจะตรวจสอบ stop_reason ก่อนนำ Response ไปใช้ หาก Response ไม่จบด้วย end_turn จะถือว่า Response ไม่สมบูรณ์และจะไม่สร้าง Test Code จาก Response ดังกล่าว แนวทางนี้ช่วยป้องกันไม่ให้นำ Test Code ที่ถูกตัดระหว่างการสร้างไปเข้าสู่ขั้นตอนการวัดผล และช่วยให้สามารถตรวจสอบสาเหตุของปัญหาจาก Response ดิบได้

2.4 ชุดข้อมูล Defects4J

Defects4J เวอร์ชัน 3.0.1 เป็นชุดข้อมูลที่ใช้ในการทดลองของโครงการ โดยประกอบด้วยโครงการภาษา Java และข้อบกพร่องจริงจำนวน 854 Active Bugs จาก 17 โครงการ ได้แก่ Lang, Math, Chart, Closure, Time, Cli, Codec, Collections, Compress, Csv, Gson, JacksonCore, JacksonDatabind, JacksonXml, Jsoup, JXPath และ Mockito

Defects4J มี Buggy Version และ Fixed Version ซึ่งสามารถนำมาใช้ตรวจสอบว่าชุดทดสอบที่สร้างขึ้นสามารถแยกความแตกต่างระหว่างโปรแกรมที่มีข้อบกพร่องและโปรแกรมที่ได้รับการแก้ไขแล้วได้หรือไม่ การใช้ Defects4J ทำให้สามารถกำหนดสภาพแวดล้อมการทดลองที่มีข้อบกพร่องจริง และสามารถประเมิน Fault Detection ได้อย่างเป็นระบบ

2.5 ตัวชี้วัดในการประเมินการสร้างกรณีทดสอบ

2.5.1 Line Coverage

Line Coverage คือสัดส่วนของบรรทัด Source Code ที่ถูกเรียกใช้งานโดย Test เมื่อเทียบกับจำนวนบรรทัดทั้งหมดที่สามารถวัดได้

2.5.2 Branch Coverage

Branch Coverage ใช้ประเมินว่าการแตกแขนงของโปรแกรมได้รับการทดสอบมากน้อยเพียงใด

2.5.3 Condition Coverage

Condition Coverage พิจารณาการประเมินเงื่อนไขภายในส่วนที่มีการแตกแขนงของโปรแกรม

2.5.4 Fault Detection Rate

Fault Detection Rate ใช้ประเมินความสามารถของ Test ที่สร้างขึ้นในการตรวจจับข้อบกพร่องจริง โดยพิจารณาจาก Bug ที่ Test สามารถทำให้ Buggy Version ล้มเหลว แต่ Fixed Version ผ่านได้

บทที่ 3

วิธีดำเนินการทดลอง

3.1 ภาพรวมการดำเนินการทดลอง

การทดลองของโครงการแบ่งออกเป็น 4 แนวทาง ได้แก่ Automated Test Case Generation Algorithms จำนวน 2 แนวทาง และ AI-Assisting Tools จำนวน 2 แนวทาง โดยในส่วน AI-Assisting Tools ใช้ Claude Sonnet 5.5 และ Gemini 3.8 Flash ผ่าน KCU IntelSphere API ส่วนอัลกอริทึมสร้างกรณีทดสอบอัตโนมัติประกอบด้วย Whole Test Suite Generation (WTS) ด้วย EvoSuite และ Coverage-guided Fuzzing

แต่ละแนวทางจะสร้าง Test Suite หรือ Test Case จาก Bug ของ Defects4J จากนั้นนำ Test ที่ได้ไปประเมินกับ Buggy Version และ Fixed Version เพื่อพิจารณาความสามารถในการตรวจจับข้อบกพร่อง และวัด Test Coverage ตามเกณฑ์ที่กำหนด

3.2 การเตรียมข้อมูลการทดลอง

3.2.1 การเตรียมชุดข้อมูล Defects4J การทดลองใช้ชุดข้อมูล Defects4J เวอร์ชัน 3.0.1 โดยแต่ละ Bug จะต้องสามารถ Checkout Buggy Version และ Fixed Version เพื่อใช้ในการประเมิน Test ที่สร้างขึ้น

3.2.2 การเตรียม Buggy Version และ Fixed Version สำหรับการประเมิน Fault Detection จะนำ Test เดียวกันไปทดสอบกับทั้ง Buggy Version และ Fixed Version เพื่อพิจารณาว่า Test สามารถแยกความแตกต่างของโปรแกรมทั้งสองเวอร์ชันได้หรือไม่

3.2.3 การเตรียมสภาพแวดล้อมการทดลอง การทดลองมีการใช้ Java ต่างเวอร์ชันในแต่ละส่วน โดย API Client ใช้ Java 17 ขณะที่ Defects4J และ Coverage Workflow ใช้ Java 11 จึงจำเป็นต้องควบคุม Environment ให้ถูกต้อง

3.3 การเตรียมอัลกอริทึมการสร้างกรณีทดสอบ

3.3.1 การเตรียม Whole Test Suite Generation สำหรับ WTS ได้จัดทำกระบวนการทดลองด้วย EvoSuite และ Shell Script เพื่อให้สามารถดำเนินการกับ Bug ใน Defects4J อย่างเป็นระบบ ขั้นตอนประกอบด้วย

1. ตรวจสอบว่า Bug มีผลการทดลองอยู่แล้วหรือไม่
2. Checkout Buggy Version
3. Compile Project
4. ตรวจสอบ Modified Classes

5. Export Compile Classpath
6. เรียก EvoSuite
7. สร้าง Test Suite
8. บันทึกสถานะและเวลาการทำงาน
9. ทำความสะอาด Checkout
10. ดำเนินการ Bug ถัดไป

ระบบถูกออกแบบให้สามารถ Resume การทดลองได้ และรองรับการใช้งานร่วมกับ tmux เพื่อให้การทดลองจำนวนมากสามารถดำเนินต่อไปได้แม้ปิด Terminal

3.3.2 การตั้งค่า EvoSuite ในการทดลองนี้ไม่ได้ใช้ EvoSuite เป็นโปรแกรมแยกที่ดาวน์โหลดมาโดยตรง แต่ใช้ evosuite-current.jar ที่มากับสภาพแวดล้อม Defects4J เพื่อหลีกเลี่ยงปัญหาความเข้ากันได้ของ Classpath ที่พบจากการใช้ EvoSuite รุ่นอื่น

Configuration ของการทดลอง Round 1 กำหนดงบเวลา 120 วินาทีต่อ Bug ใช้ Random Seed เท่ากับ 1 และใช้คลาสที่ได้รับการแก้ไขของ Bug เป็น Target Class

เป้าหมายการค้นหา ได้แก่ Branch, Line, Exception และ Weak Mutation

3.3.3 การเตรียม Coverage-guided Greybox Fuzzing ในส่วนของ Fuzzing มีการศึกษาและออกแบบกระบวนการสำหรับนำ Coverage-guided Greybox Fuzzing มาประยุกต์ใช้กับ Unit Testing โดยมีองค์ประกอบสำคัญ ได้แก่

- Seed Corpus
- Seed Selection
- Mutation
- Execution
- Coverage Feedback
- Crash Detection
- Test Minimization
- JUnit Test Export

3.3.4 การกำหนด Configuration ของ Fuzzing การออกแบบ Configuration เสนอให้ทดลองหลายระดับเพื่อศึกษาผลของ Budget และรูปแบบการสร้าง Input ได้แก่

Parameter	Configuration
Budget	30, 60, 120 วินาที
Seed Corpus	Empty / Boundary / Existing Tests
Mutation	Basic / Basic + Splice + Dictionary
Power Schedule	Uniform / Coverage-new
Repetition	5 ครั้งต่อ Configuration
Random Seed	กำหนดคงที่

ตารางที่ 2 การกำหนดค่า Configuration สำหรับการทดลอง Coverage-guided Greybox Fuzzing

ตัวชี้วัดที่เสนอประกอบด้วย Line Coverage, Branch Coverage, Fault Detection Rate, Mutation Score, จำนวน Test, ขนาด Test Suite, เวลาในการสร้าง Test, เวลาที่ Coverage ถึงจุดอิ่มตัว และ Flaky Rate

3.4 การเตรียม Generative AI

3.4.1 การเรียกใช้งาน KKU IntelSphere API การเรียก API ใช้โปรแกรม Java ผ่าน Maven และ Java 17 โดยกำหนด max_tokens = 4096 สำหรับทั้งสองโมเดล

3.4.2 การเตรียม Claude Sonnet 5.5 Claude Sonnet 5.5 ถูกนำมาใช้สร้าง Unit Test ผ่าน KKU IntelSphere API โดยใช้ Prompt ที่สร้างจากข้อมูลของแต่ละ Bug

3.4.3 การเตรียม Gemini 3.8 Flash Gemini 3.8 Flash ถูกนำมาใช้สร้าง Unit Test ผ่าน KKU IntelSphere API เช่นเดียวกับ Claude แต่มีการเพิ่ม EFFICIENCY NOTE เพื่อควบคุมการใช้ Thinking Token

3.5 การสร้างกรณีทดสอบ

3.5.1 การสร้าง Test ด้วย WTS EvoSuite ถูกใช้เพื่อสร้าง JUnit Test Suite โดยใช้กระบวนการ Search-Based Software Testing และ Whole Test Suite Generation

3.5.2 การสร้าง Test ด้วย Fuzzing Fuzzing เริ่มต้นจาก Seed Corpus จากนั้นเลือก Seed และสร้าง Input ใหม่ด้วย Mutation Operators เมื่อนำ Input ไป Execute แล้วพบ Coverage ใหม่ Input ดังกล่าวจะถูกเก็บกลับเข้า Corpus และสามารถนำมาใช้ในรอบต่อไป

3.5.3 การสร้าง Test ด้วย Claude สำหรับส่วน Generative AI ระบบจะสร้าง Prompt จากข้อมูลของแต่ละ Bug แล้วส่งไปยังโมเดลผ่าน KGU IntelSphere API โดยกำหนด max_tokens = 4096 ทั้งสองโมเดล

3.5.4 การสร้าง Test ด้วย Gemini Gemini ใช้กระบวนการเดียวกับ Claude แต่ Prompt มี EFFICIENCY NOTE เพิ่มเติมเพื่อควบคุมการใช้ Token การสร้าง Test จะเป็นการทดลองแบบหนึ่งครั้งต่อ Bug โดยไม่มีการแก้ไข Test ที่โมเดลสร้างขึ้นและไม่มีการเรียก API ซ้ำเพื่อสร้าง Test ใหม่เมื่อผลลัพธ์ไม่เป็นไปตามที่ต้องการ

3.6 การดำเนินการทดสอบ

ขั้นตอนการทำงานของส่วน Generative AI สามารถสรุปได้ดังนี้

1. เตรียม Buggy Version และ Fixed Version ของ Bug จาก Defects4J
2. ดึงข้อมูล Bug, Trigger Test และ Diff ของ Source Code
3. สร้าง Prompt ด้วย PromptGenerator
4. ส่ง Prompt ไปยัง Generative AI ผ่าน KGU IntelSphere API
5. บันทึก Response ที่ได้รับ
6. ดึง Test Code จาก Response
7. ตรวจสอบและเตรียม Test สำหรับการทดลอง
8. Compile และรัน Test บน Buggy Version
9. Compile และรัน Test บน Fixed Version
10. พิจารณาว่า Test สามารถตรวจจับ Bug ได้หรือไม่
11. วัด Line Coverage และ Condition Coverage
12. บันทึกผลการทดลองลงในไฟล์ CSV และสรุปผลของแต่ละโครงการ

3.7 การวัด Test Coverage

3.7.1 การวัด Line Coverage Line Coverage คือสัดส่วนของบรรทัด Source Code ที่ถูกเรียกใช้งานโดย Test เมื่อเทียบกับจำนวนบรรทัดทั้งหมดที่สามารถวัดได้ โดยในการทดลองนี้วัด Coverage บน Fixed Version และพิจารณาเฉพาะคลาสที่ได้รับการแก้ไขจาก Bug นั้น สามารถคำนวณได้ดังนี้

$$\text{Line Coverage} = \text{Covered Lines} / \text{Total Lines} \times 100$$

ค่า Line Coverage ที่ได้จึงไม่ได้หมายถึง Coverage ของทั้งโครงการ แต่เป็น Coverage ของคลาสที่เกี่ยวข้องกับ Bug ที่กำลังประเมินเท่านั้น

3.7.2 การวัด Branch Coverage Branch Coverage ใช้ประเมินว่าการแตกแขนงของโปรแกรมได้รับการทดสอบมากน้อยเพียงใด

3.7.3 การวัด Condition Coverage Condition Coverage พิจารณาการประเมินเงื่อนไขภายในส่วนที่มีการแตกแขนงของโปรแกรม

ในการทดลอง Generative AI ใช้ Line Coverage และ Condition Coverage เป็นตัวชี้วัดหลักเพิ่มเติมจาก Fault Detection Rate เพื่อแสดงให้เห็นว่า Test ที่สร้างขึ้นสามารถเข้าถึงส่วนต่าง ๆ ของ Source Code ได้มากน้อยเพียงใด

อย่างไรก็ตาม Coverage ไม่ได้หมายความว่า Test สามารถตรวจจับ Bug ได้เสมอไป เนื่องจาก Test อาจสามารถ Execute Code ได้เป็นจำนวนมากแต่ไม่มี Assertion หรือไม่มีเงื่อนไขที่สามารถแสดงความแตกต่างระหว่าง Buggy Version และ Fixed Version ได้ ดังนั้นจึงใช้ FDR ร่วมกับ Coverage ในการประเมิน

3.8 การวัดความสามารถในการตรวจจับข้อบกพร่อง

3.8.1 เกณฑ์การตัดสิน Fault Detection หลักการสำคัญของการประเมินคือ Test ที่สร้างขึ้นต้องสามารถแยกความแตกต่างระหว่าง Buggy Version และ Fixed Version ได้ โดยกำหนดว่า Test สามารถตรวจจับ Bug ได้เมื่อเป็นไปตามเงื่อนไขทั้ง 4 ข้อ ได้แก่

1. Compile บน Buggy Version ได้
2. Compile บน Fixed Version ได้
3. Test ล้มเหลวบน Buggy Version
4. Test ผ่านบน Fixed Version

การกำหนดเงื่อนไขทั้ง 4 ข้อช่วยลดความเสี่ยงในการนับ Test ที่ล้มเหลวจากสาเหตุอื่น เช่น Test เขียนผิด การเรียก API ไม่ถูกต้อง หรือปัญหาจาก Environment เป็น Test ที่สามารถตรวจจับ Bug ได้

3.8.2 การคำนวณ Fault Detection Rate กำหนดอัตราการตรวจจับข้อบกพร่อง (Fault Detection Rate: FDR) ดังนี้

$$FDR = \text{จำนวน Bug ที่ตรวจจับได้} / \text{จำนวน Bug ที่สามารถวัดผลได้} \times 100$$

3.8.3 การจัดกลุ่มผลการทดลอง ในการจัดกลุ่มผลการทดลอง แบ่งออกเป็น 4 กลุ่มหลัก ได้แก่

ตารางที่ 3 การจัดกลุ่มผลการทดลองตามความสามารถในการตรวจจับข้อบกพร่อง

กลุ่ม	เงื่อนไข
Detected	Compile ผ่านทั้งสอง Version, Test ล้มเหลวบน Buggy และผ่านบน Fixed
Not Detected	Test ผ่านทั้งสอง Version, ล้มเหลวทั้งสอง Version หรือ Compile ไม่ผ่านเนื่องจากปัญหาของ Test
วัดไม่ได้	ปัญหาเกิดจาก Helper, Script หรือ Environment ที่ไม่ใช่ความผิดของ Test
ตอบไม่จบ	โมเดลไม่สามารถสร้าง Test Code ได้เนื่องจาก Response สิ้นสุดที่ max_tokens

กรณี “วัดไม่ได้” และ “ตอบไม่จบ” จะไม่ถูกนำไปรวมในตัวหารของ FDR หลัก แต่จะแสดงจำนวนแยกต่างหาก เพื่อให้สามารถพิจารณาผลกระทบของปัญหาดังกล่าวได้อย่างชัดเจน นอกจากนี้สามารถนำทั้งสองกลุ่มมานับเป็นความล้มเหลวเพิ่มเติมในการวิเคราะห์ความไว (Sensitivity Analysis) ได้

3.9 การเปรียบเทียบผลการทดลอง

การเปรียบเทียบแนวทางทั้ง 4 วิธีจะพิจารณาจากลักษณะของวิธีการสร้าง Test และตัวชี้วัดที่กำหนด ได้แก่ Coverage, Fault Detection, จำนวน Test และเวลาในการสร้าง Test อย่างไรก็ตาม การเปรียบเทียบเชิงตัวเลขจะดำเนินการเฉพาะกรณีที่มีชุดข้อมูลและเงื่อนไขการทดลองที่สามารถเปรียบเทียบกันได้

บทที่ 4

ผลการทดลอง

4.1 ผลการทดลอง Whole Test Suite Generation

การทดลอง Whole Test Suite Generation (WTS) ดำเนินการโดยใช้เครื่องมือ EvoSuite ร่วมกับ Shell Script เพื่อสร้างชุดทดสอบสำหรับข้อบกพร่องในชุดข้อมูล Defects4J โดยกระบวนการทดลองถูกออกแบบให้สามารถดำเนินการกับข้อบกพร่องจำนวนมากได้อย่างเป็นระบบ และสามารถดำเนินการต่อจากจุดที่หยุดไว้ได้ในกรณีที่การทดลองใช้ระยะเวลานาน นอกจากนี้ยังรองรับการทำงานร่วมกับ tmux เพื่อให้กระบวนการทดลองสามารถทำงานต่อเนื่องได้โดยไม่ขึ้นอยู่กับการใช้งาน Terminal ตลอดระยะเวลาการทดลอง

การทดลองกำหนดให้ EvoSuite สร้างชุดทดสอบโดยใช้เป้าหมายด้าน Branch, Line, Exception และ Weak Mutation ตามค่าการกำหนดที่ระบุไว้ในวิธีดำเนินการทดลอง และใช้ Random Seed ที่กำหนดไว้เพื่อควบคุมเงื่อนไขของการทดลองให้สามารถทำซ้ำได้

ผลการประเมิน WTS พิจารณาจากจำนวนข้อบกพร่องที่สามารถสร้างชุดทดสอบได้สำเร็จ ค่า Test Coverage ความสามารถในการตรวจจับข้อบกพร่อง และระยะเวลาในการสร้างชุดทดสอบ โดยผลการทดลองในแต่ละรอบจะนำมาใช้สำหรับวิเคราะห์ประสิทธิภาพของ Whole Test Suite Generation และเปรียบเทียบกับแนวทางการสร้างชุดทดสอบรูปแบบอื่นในส่วนการวิเคราะห์ผล

4.2 ผลการทดลอง Coverage-guided Greybox Fuzzing

การทดลอง Coverage-guided Greybox Fuzzing มุ่งเน้นการสร้างข้อมูลทดสอบโดยอาศัยข้อมูล Coverage ที่ได้จากการทำงานของโปรแกรมเป็นข้อมูลย้อนกลับสำหรับควบคุมกระบวนการสร้างข้อมูลทดสอบ โดยกระบวนการประกอบด้วยกำหนด Seed Corpus การเลือก Seed การสร้างข้อมูลใหม่ด้วย Mutation การนำข้อมูลไปประมวลผล การตรวจสอบ Coverage และการคัดเลือกข้อมูลที่สามารถเพิ่ม Coverage ให้กับโปรแกรมเพื่อใช้ในการทดลองรอบถัดไป

กระบวนการทดลองประกอบด้วยขั้นตอนหลัก ได้แก่ การเตรียม Seed Corpus, Seed Selection, Mutation, Execution, Coverage Feedback, Crash Detection, Test Minimization และการแปลงข้อมูลที่ได้เป็น JUnit Test สำหรับนำไปใช้ในขั้นตอนการทดสอบ

การทดลองกำหนด Configuration สำหรับศึกษาผลของระยะเวลาในการสร้างข้อมูลและรูปแบบของข้อมูลเริ่มต้น ดังตารางที่ 4

ตารางที่ 4 การกำหนดค่า Configuration สำหรับการทดลอง Coverage-guided Greybox Fuzzing

Parameter	Configuration
Budget	30, 60 และ 120 วินาที
Seed Corpus	Empty, Boundary และ Existing Tests
Mutation	Basic และ Basic + Splice + Dictionary
Power Schedule	Uniform และ Coverage-new
Repetition	5 ครั้งต่อ Configuration
Random Seed	กำหนดค่าคงที่

การประเมินผลพิจารณาจาก Line Coverage, Branch Coverage, Fault Detection Rate, Mutation Score, จำนวน Test, ขนาดของ Test Suite, ระยะเวลาในการสร้างชุดทดสอบ, ระยะเวลาที่ Coverage เข้าสู่ภาวะอิ่มตัว และอัตราการเกิด Flaky Test เพื่อให้สามารถประเมินประสิทธิภาพของ Coverage-guided Greybox Fuzzing ได้ทั้งในด้านความครอบคลุมของโปรแกรมและความสามารถในการตรวจจับข้อบกพร่อง

ผลการทดลองของแต่ละ Configuration จะนำมาเปรียบเทียบเพื่อศึกษาผลของ Budget, Seed Corpus และ Mutation Operator ที่มีต่อประสิทธิภาพของการสร้างชุดทดสอบ รวมถึงใช้เป็นข้อมูลสำหรับเปรียบเทียบกับ Whole Test Suite Generation และ AI-Assisting Tools ในบทวิเคราะห์ผลการทดลอง

4.3 ผลการทดลอง Claude Sonnet 5.5

Claude Sonnet 5.5 ถูกนำมาใช้สร้าง Unit Test สำหรับหลาย Project ใน Defects4J โดยผลที่ดำเนินการและยืนยันในกระบวนการทดลองประกอบด้วย Codec, Csv, Gson, JXPath, Cli, JacksonXml และ Time

Project	จำนวน Bug	Bug ที่ตรวจจับได้	FDR
Codec	18	16	88.89%
Csv	16	13	81.25%
Gson	18	17	94.44%
JXPath	22	9	81.82%*
Cli	39	36	92.31%
JacksonXml	6	5	83.33%
Time	22	22	100.00%

ตารางที่ 5 ผลการตรวจจับข้อบกพร่องจากชุดทดสอบที่สร้างโดย Claude Sonnet 5.5

JXPath มี Bug ที่สามารถวัดผลได้ 11 รายการ เนื่องจากอีกส่วนหนึ่งไม่สามารถวัดผลได้จากปัญหา NoClassDefFoundError ของ DocumentLS ดังนั้นค่า FDR 81.82% คำนวณจาก Bug ที่สามารถวัดผลได้ สำหรับ Cli มีผลการวัดเพิ่มเติม โดยมี Line Coverage เฉลี่ย 42.4% และ Condition Coverage เฉลี่ย 30.5% จาก Bug ที่ Test สามารถ Compile ได้

สำหรับ JacksonXml มี Line Coverage เฉลี่ย 27.1% และ Condition Coverage เฉลี่ย 19.6%

สำหรับ Time มี Line Coverage เฉลี่ย 18.3% และ Condition Coverage เฉลี่ย 13.2% ตามข้อมูลผลการทดลองที่จัดเก็บไว้

ผลการทดลองของ Claude แสดงให้เห็นว่า Test ที่สร้างโดยโมเดลสามารถตรวจจับข้อบกพร่องจริงของ Defects4J ได้ในหลาย Project อย่างไรก็ตาม ค่า FDR แตกต่างกันระหว่าง Project เช่น Gson มี FDR สูงกว่า Codec และ Csv ขณะที่ความแตกต่างดังกล่าวไม่สามารถสรุปได้ว่าเกิดจากความสามารถของโมเดลเพียงอย่างเดียว เนื่องจากลักษณะของ Bug และโครงสร้างของแต่ละ Project แตกต่างกัน

4.4 ผลการทดลอง Gemini 3.8 Flash

Gemini 3.8 Flash ถูกนำมาใช้กับ Project ที่แตกต่างจากชุดหลักของ Claude เพื่อเพิ่มจำนวน Project ที่สามารถศึกษาได้ภายใต้ข้อจำกัดของ API Quota

ชุด Project ที่มีข้อมูลผลการทดลองในเอกสารประกอบ ได้แก่ JacksonCore, Collections, Mockito และ Compress โดยมีข้อมูลดังนี้

Project	Bug ทั้งหมด	Bug ที่ วัตได้	Detected	FDR	Avg. Line Coverage	Avg. Condition Coverage
JacksonCore	26	26	15	57.69%	19.9%	11.8%
Collections	28	28	26	92.86%	23.8%	22.4%
Mockito	38	35	28	80.00%	49.2%	34.2%
Compress	47	42	37	88.10%	32.4%	23.8%
รวม	—	131	106	80.92%	—	—

ตารางที่ 6 ผลการตรวจจับข้อบกพร่องและ Test Coverage ของ Gemini 3.8 Flash

สำหรับ Compress มีการวัดผลถึง Bug ที่ 45 จากจำนวน 47 Bug ตามข้อมูลการทดลองที่จัดทำไว้ ดังนั้นผลของ Compress ต้องพิจารณาตามจำนวน Bug ที่สามารถวัดผลได้ ไม่ควรตีความว่าเป็นผลของทั้ง 47 Bug

จากผลดังกล่าว Collections มี FDR สูงที่สุดที่ 92.86% ขณะที่ JacksonCore มี FDR ต่ำที่สุดที่ 57.69% ส่วน Mockito มี Line Coverage และ Condition Coverage เฉลี่ยสูงที่สุดในกลุ่มนี้ อย่างไรก็ตาม ค่า Coverage ที่สูงกว่าไม่ได้หมายความว่า FDR จะสูงกว่าเสมอไป ตัวอย่างเช่น Mockito มี Line Coverage และ Condition Coverage สูงกว่า Collections แต่มี FDR ต่ำกว่า Collections แสดงให้เห็นว่าการเข้าถึง Source Code และความสามารถในการตรวจจับข้อบกพร่องเป็นคนละประเด็นกัน

4.4.1 ผลการทดลอง Gemini ใน JacksonCore

ในการทดลอง Gemini กับ JacksonCore มีการสร้าง Prompt ครบทั้ง 26 Bug และเรียก Gemini สำเร็จครบ 26 รายการ โดยไม่มี API Failure

ในช่วงแรก JacksonCore-1 พบปัญหา Gemini ใช้ Token ไปกับ Thinking จนหมด ส่งผลให้ได้ stop_reason = max_tokens และไม่มี Test Code จากนั้นจึงเพิ่ม EFFICIENCY NOTE ใน Prompt หลังจากเพิ่มข้อความดังกล่าว Gemini สามารถตอบกลับได้ครบ โดยผลการทดลองที่ตรวจสอบไว้พบว่า Thinking Token อยู่ในช่วงประมาณ 276 ถึง 1,336 Token และ Response จบด้วย end_turn

ในการนำ Test ไปวัดผลยังพบว่า Test บางรายการใช้ Helper Class เช่น BaseTest และ AsyncTestBase จึงต้องจัดการ Dependency ดังกล่าวก่อนวัด Coverage เพื่อไม่ให้ Failure ของ Helper ถูกนับเป็น Failure ของ Test ที่สร้างโดย Gemini

ข้อมูลดังกล่าวแสดงให้เห็นว่าในการใช้ Generative AI สำหรับการสร้าง Unit Test จำเป็นต้องควบคุมทั้ง Prompt, Token Budget, โครงสร้าง Test และ Dependency ของ Project ไม่ใช่เพียงการส่ง Source Code ให้โมเดลสร้าง Test แล้วนำไป Execute โดยตรง

4.5 ผลการวัด Test Coverage

สำหรับผลการทดลอง Gemini 3.8 Flash ที่มีข้อมูลในปัจจุบัน ค่าเฉลี่ย Line Coverage อยู่ในช่วง 19.9%–49.2% และ Condition Coverage อยู่ในช่วง 11.8%–34.2%

สำหรับ Claude มีผลการวัดเพิ่มเติมดังนี้

- Cli มี Line Coverage เฉลี่ย 42.4% และ Condition Coverage เฉลี่ย 30.5%
- JacksonXml มี Line Coverage เฉลี่ย 27.1% และ Condition Coverage เฉลี่ย 19.6%
- Time มี Line Coverage เฉลี่ย 18.3% และ Condition Coverage เฉลี่ย 13.2%

Coverage ที่วัดเป็น Coverage ของคลาสที่เกี่ยวข้องกับ Bug ไม่ใช่ Coverage ของทั้งโครงการ

4.6 ผลการวัด Fault Detection

การประเมินความสามารถในการตรวจจับข้อบกพร่องพิจารณาจากเงื่อนไขที่กำหนดให้ชุดทดสอบต้องสามารถ Compile ได้ทั้ง Buggy Version และ Fixed Version โดยชุดทดสอบต้องทำให้ Buggy Version ล้มเหลว และ Fixed Version ผ่านการทดสอบ จึงจะถือว่าสามารถตรวจจับข้อบกพร่องได้

ผลการทดลองของ Gemini 3.8 Flash พบว่าสามารถตรวจจับข้อบกพร่องได้จำนวน 106 จาก 131 Bug ที่สามารถนำมาประเมินผลได้ คิดเป็น Fault Detection Rate (FDR) เท่ากับ 80.92% โดยเมื่อพิจารณาแยกตามโครงการ พบว่า Collections มีค่า FDR สูงที่สุดเท่ากับ 92.86% ขณะที่ JacksonCore มีค่า FDR ต่ำที่สุดเท่ากับ 57.69% ผลการทดลองดังกล่าวแสดงให้เห็นว่าความสามารถในการตรวจจับข้อบกพร่องของชุดทดสอบที่สร้างขึ้นมีความแตกต่างกันตามลักษณะของโครงการและข้อบกพร่อง

สำหรับ Claude AI Sonnet 5.5 ผลการประเมินค่า FDR จำแนกตามโครงการได้ดังตารางที่ 7

ตารางที่ 7 ค่า Fault Detection Rate (FDR) ของ Claude Sonnet 5.5 จำแนกตามโครงการ

โครงการ	Fault Detection Rate (FDR)
Codec	88.89%
Csv	81.25%
Gson	94.44%
JXPath	81.82%
Cli	92.31%
JacksonXml	83.33%
Time	100.00%

จากผลการทดลองพบว่า Time มีค่า FDR สูงที่สุดเท่ากับ 100.00% รองลงมาคือ Gson เท่ากับ 94.44% และ Cli เท่ากับ 92.31% ขณะที่ Csv มีค่า FDR ต่ำที่สุดเท่ากับ 81.25% เมื่อพิจารณาผลของ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ร่วมกัน พบว่าค่า Fault Detection Rate มีความแตกต่างกันในแต่ละโครงการ

อย่างไรก็ตาม การเปรียบเทียบค่า FDR ระหว่าง Claude AI Sonnet 5.5 และ Gemini 3.8 Flash จำเป็นต้องพิจารณาเงื่อนไขของการทดลองร่วมด้วย เนื่องจากทั้งสองแนวทางไม่ได้ถูกทดลองกับชุดโครงการและข้อบกพร่องเดียวกันทั้งหมด ดังนั้นค่า FDR ที่ได้จึงไม่สามารถใช้เป็นหลักฐานเพียงอย่างเดียวในการสรุปว่าเครื่องมือใดมีประสิทธิภาพในการตรวจจับข้อบกพร่องสูงกว่า การวิเคราะห์จึงควรพิจารณาพร้อมกับจำนวน Bug ที่สามารถวัดผลได้ ลักษณะของโครงการ Test Coverage และข้อจำกัดที่เกิดขึ้นระหว่างการทดลองด้วย

บทที่ 5

วิเคราะห์และอภิปรายผล

5.1 การเปรียบเทียบ Whole Test Suite Generation และ Coverage-guided Greybox Fuzzing

Whole Test Suite Generation (WTS) และ Coverage-guided Greybox Fuzzing เป็นแนวทางการสร้างชุดทดสอบอัตโนมัติที่มีหลักการทำงานแตกต่างกัน โดย WTS ใช้แนวคิด Search-Based Software Testing ซึ่งพิจารณา Test Suite เป็น Candidate Solution และใช้ค่า Fitness หรือ Coverage เป็นข้อมูลสำหรับประเมินและปรับปรุงชุดทดสอบ กระบวนการดังกล่าวมุ่งค้นหาชุดทดสอบที่สามารถตอบสนองเป้าหมายด้าน Coverage ที่กำหนดไว้

ในทางตรงกันข้าม Coverage-guided Greybox Fuzzing ใช้ข้อมูล Coverage ที่ได้จากการ Execute Program เป็น Feedback สำหรับคัดเลือก Input ที่สามารถเข้าถึงเส้นทางการทำงานใหม่ จากนั้นนำ Input ที่มีประโยชน์กลับมาใช้เป็น Seed สำหรับสร้างข้อมูลทดสอบในรอบถัดไป กระบวนการดังกล่าวจึงมีลักษณะเด่นในการสร้างและสำรวจ Input จำนวนมาก รวมถึงสามารถค้นหา Input ที่นำไปสู่ Edge Case หรือพฤติกรรมที่ไม่คาดคิดของโปรแกรมได้

เมื่อพิจารณาจากลักษณะการทำงาน WTS มีความเหมาะสมกับการค้นหาชุดทดสอบที่มีเป้าหมายด้าน Coverage อย่างเป็นระบบ เนื่องจากกระบวนการ Search สามารถปรับปรุงชุดทดสอบตามค่า Fitness ที่กำหนดไว้ ขณะที่ Coverage-guided Greybox Fuzzing มีจุดเด่นด้านการสำรวจ Input Space และการค้นหาเส้นทางการทำงานใหม่ผ่านการปรับเปลี่ยนข้อมูลด้วย Mutation อย่างไรก็ตาม ประสิทธิภาพของทั้งสองแนวทางขึ้นอยู่กับ Configuration ที่ใช้ในการทดลอง เช่น ระยะเวลาในการ Search หรือ Fuzzing, Seed, Mutation Operator และเป้าหมายด้าน Coverage ดังนั้นการประเมินประสิทธิภาพจึงควรพิจารณาพร้อมกันทั้ง Test Coverage, Fault Detection Rate, จำนวนชุดทดสอบ และระยะเวลาในการสร้างชุดทดสอบ โดยผลการทดลองของทั้งสองแนวทางในชุดข้อมูลที่ใช้ศึกษา ยังไม่ครอบคลุมเงื่อนไขทั้งหมด จึงไม่สามารถสรุปเชิงตัวเลขได้อย่างชัดเจนว่าแนวทางใดมีประสิทธิภาพสูงกว่า

5.2 การเปรียบเทียบ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash

ผลการทดลองของ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash แสดงให้เห็นว่าสามารถนำ AI-Assisting Tools มาใช้ในการสร้างชุดทดสอบระดับ Unit Testing จากข้อมูลของ Source Code และข้อมูลเกี่ยวกับข้อบกพร่องได้ อย่างไรก็ตาม การเปรียบเทียบประสิทธิภาพของทั้งสองเครื่องมือจำเป็นต้องพิจารณาเงื่อนไขของชุดข้อมูลที่ใช้ในการทดลอง เนื่องจาก Claude และ Gemini ถูกนำไปทดลองกับ Project และ Bug ที่แตกต่างกันภายใต้ข้อจำกัดด้าน API Quota

ตัวอย่างเช่น Gemini 3.8 Flash มีค่า Fault Detection Rate เท่ากับ 57.69% สำหรับ JacksonCore แต่ไม่สามารถนำค่าดังกล่าวมาใช้สรุปว่า Gemini มีประสิทธิภาพต่ำกว่า Claude AI Sonnet 5.5 เนื่องจาก

Claude ไม่ได้ถูกทดลองกับ JacksonCore ภายใต้เงื่อนไขเดียวกัน การเปรียบเทียบจากค่า FDR เพียงค่าเดียว จึงอาจได้รับอิทธิพลจากความแตกต่างของลักษณะ Project และ Bug ที่นำมาใช้ในการทดลอง

นอกจาก Fault Detection Rate แล้ว การประเมิน AI-Assisting Tools ควรพิจารณาความสามารถในการสร้าง Test Code ที่สามารถ Compile และ Execute ได้ ความสามารถในการสร้าง Assertion ที่สามารถตรวจจับข้อบกพร่อง ค่า Test Coverage จำนวน Test ที่สร้างขึ้น ปัญหาที่เกิดขึ้นระหว่างการสร้าง Test และทรัพยากรที่ใช้ในการเรียก API

จากผลการทดลองพบว่าการสร้างชุดทดสอบด้วย AI-Assisting Tools ไม่ได้ขึ้นอยู่กับความสามารถของโมเดลเพียงอย่างเดียว แต่ยังเกี่ยวข้องกับข้อมูลที่ใช้สร้าง Prompt, โครงสร้างของ Source Code, ข้อมูลของ Bug, Token Budget และข้อจำกัดของ API โดยเฉพาะกรณี Gemini ที่พบ Response ไม่สมบูรณ์เมื่อใช้ Token ไปถึงกระบวนการ Thinking จนถึงขีดจำกัดที่กำหนดไว้

ดังนั้น หากต้องการเปรียบเทียบ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash โดยตรง ควรกำหนดให้ทั้งสองเครื่องมือสร้างชุดทดสอบจาก Project และ Bug ชุดเดียวกัน และควบคุม Prompt, Configuration และเงื่อนไขการประเมินให้ใกล้เคียงกันมากที่สุด ซึ่งเงื่อนไขดังกล่าวอยู่นอกเหนือจากชุดการทดลองที่ดำเนินการในครั้งนี้

5.3 การเปรียบเทียบทั้ง 4 แนวทาง

การทดลองนี้ประกอบด้วย 4 แนวทาง ได้แก่ Whole Test Suite Generation (WTS), Coverage-guided Greybox Fuzzing, Claude AI Sonnet 5.5 และ Gemini 3.8 Flash โดยแต่ละแนวทางมีหลักการสร้างชุดทดสอบและลักษณะการทำงานที่แตกต่างกัน ดังแสดงในตารางที่ 8

ตารางที่ 8 การเปรียบเทียบลักษณะของแนวทางการสร้างชุดทดสอบทั้ง 4 แนวทาง

แนวทาง	หลักการ	จุดเด่น	ข้อจำกัด
WTS / EvoSuite	Search-Based Software Testing	ค้นหาและปรับปรุง Test Suite โดยใช้ Fitness และ Coverage	ใช้เวลาในการ Search และขึ้นอยู่กับ Budget ที่กำหนด
Coverage-guided Greybox Fuzzing	Coverage Feedback และ Mutation	สร้าง Input จำนวนมากและสำรวจเส้นทางการทำงานใหม่	ต้องออกแบบ Harness และกำหนด Oracle ที่เหมาะสม
Claude AI Sonnet 5.5	AI-Assisting Tool ที่ใช้ Generative AI	วิเคราะห์ Source Code และข้อมูลของ Bug เพื่อสร้าง Test Code จาก Prompt	Test ที่สร้างขึ้นอาจ Compile ไม่ผ่านหรือมี Assertion ที่ไม่เหมาะสม
Gemini 3.8 Flash	AI-Assisting Tool ที่ใช้ Generative AI	สามารถสร้าง Test Code จาก Source Code และข้อมูลของ Bug	มีข้อจำกัดด้าน Token และ API และอาจเกิด Response ที่ไม่สมบูรณ์

เมื่อพิจารณาหลักการทำงานของทั้ง 4 แนวทาง พบว่า WTS และ Coverage-guided Greybox Fuzzing อาศัยข้อมูลจากการ Execute Program และใช้ Feedback ที่ได้จากการทำงานของโปรแกรมเป็นส่วนสำคัญในการค้นหา Test ขณะที่ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ใช้ความสามารถของ Generative AI ในการวิเคราะห์ Source Code และข้อมูลบริบทของข้อบกพร่องเพื่อสร้าง Test Code ความแตกต่างดังกล่าวส่งผลให้แต่ละแนวทางมีจุดเด่นในการค้นหา Test ที่แตกต่างกัน WTS มุ่งเน้นการค้นหาชุดทดสอบตามเป้าหมายด้าน Coverage, Coverage-guided Greybox Fuzzing มุ่งเน้นการสำรวจ Input Space ผ่าน Coverage Feedback และ Mutation ส่วน AI-Assisting Tools สามารถใช้ข้อมูลเชิงบริบทของ Source Code และ Bug เพื่อสร้าง Test Code โดยตรง

ดังนั้น การประเมินประสิทธิภาพของทั้ง 4 แนวทางจึงไม่ควรพิจารณาจากค่า Coverage หรือ Fault Detection Rate เพียงตัวเดียว แต่ควรพิจารณาร่วมกับความสามารถในการตรวจจับข้อบกพร่อง จำนวน Test Coverage ระยะเวลาในการสร้าง Test ความสามารถในการ Compile และ Execute ตลอดจนข้อจำกัดและปัญหาที่เกิดขึ้นในแต่ละแนวทาง เพื่อให้การเปรียบเทียบสะท้อนประสิทธิภาพของแต่ละวิธีได้อย่างเหมาะสม

5.4 ความสัมพันธ์ระหว่าง Test Coverage และ Fault Detection

ผลการทดลองแสดงให้เห็นว่า Test Coverage และความสามารถในการตรวจจับข้อบกพร่องเป็นตัวชี้วัดที่มีความสัมพันธ์กันในบางส่วน แต่ค่า Coverage ที่สูงไม่ได้หมายความว่าชุดทดสอบจะสามารถตรวจจับข้อบกพร่องได้สูงเสมอไป เนื่องจาก Coverage ใช้แสดงระดับการเข้าถึงส่วนต่าง ๆ ของ Source Code ขณะที่ Fault Detection พิจารณาศักยภาพของชุดทดสอบในการแสดงความแตกต่างระหว่าง Buggy Version และ Fixed Version

ตัวอย่างเช่น ชุดทดสอบอาจสามารถ Execute Branch หรือ Line ของโปรแกรมได้เป็นจำนวนมาก แต่หาก Test ไม่มี Assertion ที่เหมาะสม หรือไม่มี Input ที่ทำให้เกิดพฤติกรรมที่แตกต่างระหว่าง Buggy Version และ Fixed Version ชุดทดสอบดังกล่าวอาจมี Coverage สูงแต่ไม่สามารถตรวจจับข้อบกพร่องได้ ในทางกลับกัน ชุดทดสอบที่มี Coverage ไม่สูงมากอาจสามารถตรวจจับข้อบกพร่องได้ หาก Test มี Input และ Assertion ที่สอดคล้องกับพฤติกรรมที่ผิดพลาดของโปรแกรม ดังนั้น Coverage จึงควรใช้เป็นตัวชี้วัดด้านความครอบคลุมของการทำงานร่วมกับ Fault Detection Rate แทนการใช้เป็นตัวชี้วัดเพียงอย่างเดียว จากแนวคิดดังกล่าว การประเมินคุณภาพของชุดทดสอบในการทดลองนี้จึงพิจารณาตัวชี้วัดหลายด้าน ได้แก่ Line Coverage, Branch Coverage หรือ Condition Coverage, Fault Detection Rate, จำนวน Test, ระยะเวลาในการสร้างชุดทดสอบ, ความสามารถในการ Compile และ Execute ตลอดจนคุณภาพและความเสถียรของชุดทดสอบ

การพิจารณาตัวชี้วัดหลายด้านช่วยให้สามารถประเมินชุดทดสอบได้ครอบคลุมมากขึ้น เนื่องจากชุดทดสอบที่มี Coverage สูงแต่ไม่สามารถตรวจจับข้อบกพร่อง หรือชุดทดสอบที่ตรวจจับข้อบกพร่องได้แต่ไม่สามารถ Compile และ Execute ได้อย่างถูกต้อง ล้วนไม่สามารถสะท้อนคุณภาพของชุดทดสอบได้อย่างสมบูรณ์ ดังนั้น Fault Detection Rate และ Test Coverage จึงควรถูกพิจารณาร่วมกันในการวิเคราะห์ผลการทดลองครั้งนี้

5.5 การวิเคราะห์รูปแบบความล้มเหลว

จากผลการทดลองพบว่าผลลัพธ์ของการสร้างชุดทดสอบไม่ได้จำแนกได้เพียงกรณีที่สามารถตรวจจับข้อบกพร่องและกรณีที่ไม่สามารถตรวจจับข้อบกพร่องเท่านั้น แต่ยังพบผลลัพธ์ที่เกี่ยวข้องกับการทำงานของชุดทดสอบ สภาพแวดล้อม และความสมบูรณ์ของ Response จาก AI-Assisting Tools การจำแนกลักษณะความล้มเหลวจึงมีความสำคัญต่อการตีความผล Fault Detection Rate และการวิเคราะห์ข้อจำกัดของแต่ละแนวทาง

5.5.1 Test ผ่านทั้ง Buggy และ Fixed Version กรณีที่ชุดทดสอบสามารถ Compile และ Execute ได้ทั้ง Buggy Version และ Fixed Version แต่ผลการทดสอบไม่สามารถแสดงความแตกต่าง

ระหว่างทั้งสอง Version ได้ จะถือว่าชุดทดสอบไม่สามารถตรวจจับข้อบกพร่องเป้าหมายได้ และจัดอยู่ในกลุ่ม Not Detected

กรณีดังกล่าวแสดงให้เห็นว่าการที่ Test สามารถทำงานได้สำเร็จไม่ได้หมายความว่า Test จะสามารถตรวจจับข้อบกพร่องได้ เนื่องจาก Test อาจไม่ได้ครอบคลุม Input หรือ Assertion ที่เกี่ยวข้องกับพฤติกรรมที่แตกต่างระหว่าง Buggy Version และ Fixed Version

5.5.2 Test ล้มเหลวทั้ง Buggy และ Fixed Version กรณีที่ชุดทดสอบล้มเหลวทั้ง Buggy Version และ Fixed Version ไม่สามารถใช้ผลการทดสอบเพื่อยืนยันได้ว่าความล้มเหลวเกิดจากข้อบกพร่องเป้าหมาย เนื่องจากอาจเกิดจากปัญหาของชุดทดสอบเอง หรือปัจจัยอื่นที่เกี่ยวข้องกับสภาพแวดล้อมการทดสอบ

จากการทดลองพบว่าความล้มเหลวในลักษณะดังกล่าวอาจเกี่ยวข้องกับ API, Helper, Resource หรือ Environment ดังนั้นกรณีนี้พบจึงต้องพิจารณาสาเหตุของความล้มเหลวประกอบก่อนจัดประเภทผลการทดลอง เพื่อป้องกันการนำความล้มเหลวที่ไม่ได้เกิดจาก Bug เป้าหมายมานับเป็น Fault Detection

5.5.3 Compile ไม่ผ่าน พบกรณีที่ Test Code ซึ่งสร้างโดย AI-Assisting Tools ไม่สามารถ Compile ได้ โดยสาเหตุส่วนหนึ่งเกิดจากการเรียกใช้ API หรือ Method ที่ไม่มีอยู่จริง การใช้งาน API ไม่ถูกต้อง หรือการอ้างอิง Class ที่ไม่สามารถใช้งานได้ ใน Environment ของโครงการ ตัวอย่างเช่น การเรียก Method ที่ไม่มีอยู่ใน Library หรือการอ้างอิง Class ที่ไม่ได้อยู่ใน Classpath ของการทดลอง กรณีที่ Compile ไม่ผ่านเนื่องจากข้อผิดพลาดของ Test Code จะถูกจัดอยู่ในกลุ่ม Not Detected เนื่องจากชุดทดสอบไม่สามารถนำไปใช้ตรวจสอบ Bug ได้ ส่วนกรณีที่สาเหตุเกิดจาก Helper, Classpath หรือ Environment ของการทดลอง จะจัดอยู่ในกลุ่ม “วัดไม่ได้” เพื่อแยกออกจากข้อผิดพลาดที่เกิดจาก Test Code โดยตรง

5.5.4 Response ไม่สมบูรณ์ ในการทดลอง Gemini 3.8 Flash มีการกำหนดค่า max_tokens เท่ากับ 4096 Token โดยกระบวนการคิดของโมเดลใช้ทรัพยากร Token ภายใต้อัตราจำกัดดังกล่าวด้วยการทดลองพบว่าในบางกรณีโมเดลใช้ Token จนถึงขีดจำกัดก่อนที่จะสร้าง Test Code ได้อย่างสมบูรณ์ ส่งผลให้ Response สิ้นสุดด้วย stop_reason = max_tokens และไม่สามารถนำผลลัพธ์ดังกล่าวไปใช้ในการประเมินชุดทดสอบได้

สำหรับโครงการ Compress พบกรณี Response ไม่สมบูรณ์จำนวน 2 Bug ได้แก่ Bug 20 และ Bug 23 โดยทั้งสองกรณี Response สิ้นสุดด้วย max_tokens และมีจำนวน output_tokens เกือบเคียงกับขีดจำกัด 4096 Token แม้ว่าจะมีการเพิ่ม EFFICIENCY NOTE ใน Prompt เพื่อควบคุมการใช้ Token แล้วก็ตาม

กรณี Response ไม่สมบูรณ์ถูกแยกออกจากผลการตรวจจับข้อบกพร่อง เนื่องจากไม่มี Test Code ที่สมบูรณ์เพียงพอสำหรับนำไป Compile และ Execute นอกจากนี้ การทดลองไม่ได้เรียก API เข้าเพื่อสร้าง Test ใหม่ และไม่นำ Code ที่ถูกตัดระหว่างการสร้างไปใช้ในการประเมิน ดังนั้นกรณีดังกล่าวจึงถูกจัดเป็น “ตอบไม่จบ”

และไม่ถูกนำมารวมในตัวหารของ Fault Detection Rate หลัก เพื่อไม่ให้ผลการประเมินสะท้อนความสามารถในการตรวจจับข้อบกพร่องจากข้อมูลที่ไม่สมบูรณ์ของโมเดล

5.6 อภิปรายผลการทดลอง

จากผลการทดลองพบว่า AI-Assisting Tools สามารถนำมาใช้สร้างชุดทดสอบระดับ Unit Testing ที่สามารถตรวจจับข้อบกพร่องจริงใน Defects4J ได้ โดยเฉพาะ Gemini 3.8 Flash ซึ่งสามารถตรวจจับข้อบกพร่องได้ 106 จาก 131 Bug ที่สามารถวัดผลได้ คิดเป็น Fault Detection Rate เท่ากับ 80.92% ผลดังกล่าวแสดงให้เห็นว่า AI-Assisting Tools ที่ใช้ความสามารถของ Generative AI มีศักยภาพในการสร้างชุดทดสอบที่สามารถนำไปใช้ตรวจจับข้อบกพร่องจริงได้

อย่างไรก็ตาม ค่า Fault Detection Rate ที่ได้ไม่สามารถใช้เป็นหลักฐานในการสรุปว่า AI-Assisting Tools สามารถทดแทน Automated Test Case Generation Algorithms ได้โดยตรง เนื่องจากการทดลองของแต่ละแนวทางมีเงื่อนไขแตกต่างกัน โดยการทดลองของ AI-Assisting Tools ยังครอบคลุม Project และ Bug ไม่ครบทุกโครงการใน Defects4J และมีข้อจำกัดด้าน API Quota ขณะที่ WTS และ Coverage-guided Greybox Fuzzing ใช้กระบวนการและ Configuration ที่แตกต่างกัน ดังนั้นการเปรียบเทียบประสิทธิภาพระหว่างแนวทางจึงต้องพิจารณาเงื่อนไขของการทดลองประกอบด้วย

สำหรับ Whole Test Suite Generation จุดเด่นอยู่ที่การใช้ Search-Based Software Testing ร่วมกับ Execution Feedback เพื่อปรับปรุง Test Suite ตามเป้าหมายที่กำหนด กระบวนการดังกล่าวเหมาะกับการค้นหาชุดทดสอบที่สามารถเพิ่ม Coverage ได้อย่างเป็นระบบ อย่างไรก็ตาม การค้นหาต้องอาศัยการ Execute โปรแกรมและทรัพยากรในการประมวลผลตาม Search Budget ที่กำหนด โดยเฉพาะเมื่อดำเนินการกับข้อบกพร่องจำนวนมาก อาจส่งผลกระทบต่อระยะเวลาและทรัพยากรที่ใช้ในการทดลอง Coverage-guided Greybox Fuzzing มีจุดเด่นในการสร้าง Input จำนวนมากและใช้ Coverage Feedback เพื่อค้นหาเส้นทางการทำงานใหม่ ทำให้มีศักยภาพในการค้นหา Edge Case หรือ Input ที่ผู้ทดสอบอาจไม่ได้กำหนดไว้ล่วงหน้า อย่างไรก็ตาม การนำ Fuzzing มาประยุกต์ใช้กับ Unit Testing จำเป็นต้องกำหนด Harness และกลไกสำหรับตรวจสอบความถูกต้องของผลลัพธ์ เนื่องจาก Fuzzer ไม่สามารถระบุได้โดยตรงว่าผลลัพธ์จาก Input แต่ละรายการถูกต้องหรือไม่ โดยเฉพาะกรณีที่ต้องการประเมินความสามารถในการตรวจจับข้อบกพร่องของโปรแกรม

ในส่วนของ AI-Assisting Tools มีข้อได้เปรียบในการวิเคราะห์ Source Code และข้อมูลบริบทของข้อบกพร่อง เช่น Class, Method, API และ Diff ของ Source Code แล้วนำข้อมูลดังกล่าวมาใช้สร้าง Test Code โดยตรง ลักษณะดังกล่าวช่วยให้สามารถสร้าง Test ที่มีโครงสร้างใกล้เคียงกับ Test ที่พัฒนาโดยมนุษย์ได้ อย่างไรก็ตาม ผลการทดลองพบข้อจำกัดหลายประการ ได้แก่ Test Code ที่ Compile ไม่ผ่าน การ

ทดสอบที่ล้มเหลวทั้ง Buggy Version และ Fixed Version ความไม่สมบูรณ์ของ Response และความแตกต่างของผลลัพธ์ระหว่างแต่ละ Project

เมื่อพิจารณาผลการทดลองโดยรวมจึงพบว่าแต่ละแนวทางมีจุดเด่นและข้อจำกัดแตกต่างกัน WTS มีจุดเด่นด้านการค้นหาและปรับปรุง Test Suite ด้วย Search และ Coverage Feedback ขณะที่ Coverage-guided Greybox Fuzzing มีจุดเด่นด้านการสำรวจ Input Space และการค้นหา Edge Case ส่วน AI-Assisting Tools มีความสามารถในการวิเคราะห์ Source Code และสร้าง Test Code จากข้อมูลบริบทของ Bug ดังนั้น การเลือกใช้แนวทางใดแนวทางหนึ่งจึงควรพิจารณาตามวัตถุประสงค์และเงื่อนไขของการทดสอบ

จากข้อสังเกตดังกล่าว สามารถนำไปสู่แนวทางการศึกษาการทำงานร่วมกันระหว่างวิธีการต่าง ๆ เช่น การใช้ AI-Assisting Tools เพื่อสร้าง Test เบื้องต้น แล้วใช้ Search-Based Software Testing หรือ Fuzzing เพื่อค้นหา Input เพิ่มเติม รวมถึงการใช้ Coverage Feedback เพื่อวิเคราะห์และปรับปรุงชุดทดสอบที่สร้างโดย AI อย่างไรก็ตาม แนวทางดังกล่าวยังไม่ได้ถูกนำมาทดลองในโครงการนี้ จึงถือเป็นข้อเสนอแนะสำหรับการศึกษาค่อยๆ และไม่สามารถสรุปว่าเป็นผลที่ได้รับการยืนยันจากการทดลองครั้งนี้ได้

บทที่ 6

ข้อจำกัด ปัญหา และความสามารถในการทำซ้ำการทดลอง

6.1 ข้อจำกัดของการทดลอง

การทดลองทั้ง 4 แนวทางมีข้อจำกัดหลายประการที่อาจส่งผลกระทบต่อความดีความและการเปรียบเทียบผลการทดลอง โดยมีรายละเอียดดังนี้

1. การสร้างชุดทดสอบสำหรับแต่ละ Bug ดำเนินการเพียงหนึ่งครั้ง โดยไม่มีการสร้าง Test ใหม่จาก Prompt เดิมหรือปรับปรุง Test ที่สร้างขึ้นด้วยวิธีการอื่น ดังนั้น ผลการทดลองจึงยังไม่สามารถสะท้อนความแปรปรวนของผลลัพธ์ที่อาจเกิดขึ้นจากการสร้างชุดทดสอบหลายครั้งได้
2. Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ถูกนำไปทดลองกับชุด Project และ Bug ที่แตกต่างกัน เนื่องจากข้อจำกัดด้าน API Quota ส่งผลให้ค่า Fault Detection Rate ของทั้งสองเครื่องมือไม่สามารถนำมาเปรียบเทียบโดยตรงเพื่อสรุปประสิทธิภาพของโมเดลได้ โดยจำเป็นต้องพิจารณาความแตกต่างของชุดข้อมูลและเงื่อนไขการทดลองร่วมด้วย
3. Prompt ที่ใช้กับ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash มีความแตกต่างกันบางส่วน โดย Gemini มีการเพิ่ม EFFICIENCY NOTE เพื่อควบคุมการใช้ Token และลดปัญหา Response ไม่สมบูรณ์จากกระบวนการ Thinking ขณะที่ Claude ไม่มีข้อความดังกล่าว ความแตกต่างของ Prompt จึงอาจมีผลต่อรูปแบบและคุณภาพของ Test Code ที่สร้างขึ้น
4. การวัด Test Coverage พิจารณาเฉพาะคลาสที่ได้รับการแก้ไขของ Bug บน Fixed Version ไม่ได้ครอบคลุม Source Code ทั้งหมดของโครงการ ดังนั้นค่า Coverage ที่รายงานจึงแสดงระดับความครอบคลุมของส่วนที่เกี่ยวข้องกับ Bug ที่นำมาทดลอง และไม่ควรถือความว่าเป็น Coverage ของทั้งโครงการ
5. ผลการวัด Coverage บางรายการมีค่า 0.0% ซึ่งในบางกรณีไม่สามารถแยกได้อย่างสมบูรณ์ว่าค่าดังกล่าวเกิดจาก Test ไม่สามารถเข้าถึง Source Code หรือเกิดจากไม่มีข้อมูลที่สามารถนำมาวัด Coverage ได้ ส่งผลให้การคำนวณค่าเฉลี่ย Coverage ของบางโครงการอาจได้รับผลกระทบจากกรณีดังกล่าว
6. การประเมินผลของ Mockito, JacksonCore และ Collections ใช้สคริปต์รุ่นแรก ขณะที่ Compress ใช้สคริปต์ gemini_eval2.sh ซึ่งได้รับการปรับปรุงให้สามารถบันทึกผลแยกตาม Bug และรวมผลการทดลองโดยอัตโนมัติ แม้เกณฑ์การพิจารณา Fault Detection จะเหมือนกัน แต่ยังไม่มีการทำ Regression Test เพื่อยืนยันว่าการทำงานของสคริปต์ทั้งสองรุ่นให้ผลลัพธ์เหมือนกันในทุกกรณี

7. ข้อจำกัดด้าน API Quota ส่งผลต่อจำนวน Bug และจำนวน Project ที่สามารถนำมาทดลองด้วย AI-Assisting Tools ทำให้ไม่สามารถดำเนินการทดลองได้ครบทุกโครงการใน Defects4J ตามขอบเขตที่กำหนดไว้
8. กรณีที่ไม่สามารถวัดผลได้และกรณี Response ไม่สมบูรณ์ถูกแยกออกจากตัวหารของ Fault Detection Rate หลัก เพื่อไม่ให้ปัจจัยจาก Environment และข้อจำกัดของ API ถูกนำมาปะปนกับความสามารถในการตรวจจับข้อบกพร่องของ Test อย่างไรก็ดีตาม การไม่นับกรณีดังกล่าวอาจทำให้ค่า FDR สูงกว่ากรณีที่น่าผลลัพธ์เหล่านี้นับเป็นความล้มเหลว จึงควรพิจารณาผลการวิเคราะห์ความไว้วางใจด้วย
9. ผลการทดลองบางกรณียังจำเป็นต้องตรวจสอบสาเหตุของความล้มเหลวเพิ่มเติม โดยเฉพาะกรณีที่ Test ล้มเหลวทั้ง Buggy Version และ Fixed Version หรือกรณีที่ Test Compile ไม่ผ่าน เพื่อแยกสาเหตุระหว่างข้อผิดพลาดของ Test Code กับปัญหาที่เกิดจาก Helper, Classpath หรือ Environment
10. ผลการทดลองของโครงการ Compress ยังไม่ครอบคลุม Bug ทั้งหมด เนื่องจาก Bug 46 และ Bug 47 ยังไม่ได้ดำเนินการ ดังนั้นผลที่นำเสนอสำหรับ Compress จึงเป็นผลจาก Bug ที่สามารถดำเนินการและวัดผลได้ในช่วงการทดลอง
11. การทดลอง Gemini 3.8 Flash ยังไม่ครอบคลุมโครงการ Chart, Lang, Jsoup, Math, JacksonDatabind และ Closure เนื่องจากข้อจำกัดด้าน API Quota และระยะเวลาในการดำเนินการ ดังนั้นผลการทดลองที่ได้จึงไม่สามารถใช้เป็นตัวแทนของความสามารถของ AI-Assisting Tools บนชุดข้อมูล Defects4J ทั้งหมดได้

6.2 ปัญหาและอุปสรรคในการดำเนินการทดลอง

6.2.1 ปัญหาด้าน Defects4J ในการดำเนินการทดลองกับ Defects4J เวอร์ชัน 3.0.1 พบว่าคำสั่งบางส่วนจาก Workflow ของ Defects4J รุ่นก่อนหน้านี้ไม่สามารถนำมาใช้ได้โดยตรง เช่น `defects4j gen_tests` ส่งผลให้การสร้างชุดทดสอบด้วย Whole Test Suite Generation จำเป็นต้องเรียกใช้ EvoSuite โดยตรง และพัฒนากระบวนการสำหรับจัดการ Checkout, Compile และ Classpath ให้สอดคล้องกับสภาพแวดล้อมของการทดลอง

6.2.2 ปัญหาด้าน Java Version การทดลองมีการใช้ Java ต่างเวอร์ชันในแต่ละส่วนของระบบ โดย API Client สำหรับเรียกใช้ AI-Assisting Tools ใช้ Java 17 ขณะที่ Defects4J และกระบวนการวัด Coverage ใช้ Java 11 ความแตกต่างดังกล่าวจำเป็นต้องควบคุม Java Version ของแต่ละขั้นตอนให้ถูกต้องเพื่อป้องกันปัญหาความเข้ากันได้ระหว่างเครื่องมือและโครงการที่นำมาทดลอง

6.2.3 ปัญหาด้าน API Quota การเรียกใช้ AI-Assisting Tools ผ่าน KCU IntelSphere API มีข้อจำกัดด้าน API Quota ซึ่งส่งผลโดยตรงต่อจำนวน Request และจำนวน Bug ที่สามารถนำมาทดลองได้ ทำให้ไม่สามารถดำเนินการทดลองกับทุก Project ใน Defects4J ได้ภายในเงื่อนไขของการทดลอง

6.2.4 ปัญหาด้าน Response ไม่สมบูรณ์ ในการทดลอง Gemini 3.8 Flash พบว่าบาง Response สิ้นสุดลงก่อนที่จะสร้าง Test Code ได้อย่างสมบูรณ์ เนื่องจาก Token ที่ใช้ในกระบวนการ Thinking รวมอยู่ภายใต้ข้อจำกัด max_tokens ส่งผลให้ Response สิ้นสุดด้วย stop_reason = max_tokens และไม่สามารถนำ Test Code ดังกล่าวไปใช้ในขั้นตอนการ Compile และประเมิน Fault Detection ได้

6.2.5 ปัญหาด้านการ Compile ของ Test พบว่า Test Code บางรายการที่สร้างโดย AI-Assisting Tools ไม่สามารถ Compile ได้ เนื่องจากการอ้างอิง Class, Method หรือ API ที่ไม่มีอยู่จริง หรือไม่สามารถใช้งานได้ ใน Environment ของโครงการ ปัญหาดังกล่าวส่งผลให้ Test ไม่สามารถเข้าสู่ขั้นตอนการ Execute และการประเมินความสามารถในการตรวจจับข้อบกพร่องได้

6.3 ความสามารถในการทำซ้ำการทดลอง

เพื่อสนับสนุนความสามารถในการตรวจสอบและทำซ้ำการทดลอง โครงการนี้ได้จัดเก็บองค์ประกอบที่เกี่ยวข้องกับกระบวนการทดลองอย่างเป็นระบบ ได้แก่ Source Code ของเครื่องมือ, Shell Script, Configuration, Prompt, Generated Test, Raw API Response, Result CSV, Coverage Result, Log และ README

สำหรับการทดลอง AI-Assisting Tools ได้จัดเก็บ Prompt แยกตาม Project และ Bug เพื่อให้สามารถตรวจสอบข้อมูลที่ใช้ในการสร้าง Test ของแต่ละ Bug ได้

นอกจาก Prompt แล้ว ยังมีการจัดเก็บ Response ที่ได้รับจาก API และ Test Code ที่สร้างขึ้น แยกตาม Project และ Bug เพื่อให้สามารถตรวจสอบย้อนกลับกระบวนการสร้างชุดทดสอบได้ ผลการทดลองถูกจัดเก็บในรูปแบบตารางข้อมูล โดยประกอบด้วยข้อมูลสำคัญ เช่น

- bug
- test_files
- methods
- compile_buggy
- test_buggy
- compile_fixed
- test_fixed
- fault_detected
- lines_covered
- lines_total

- line_pct
- cond_covered
- cond_total
- cond_pct
- note

ข้อมูลดังกล่าวช่วยให้สามารถตรวจสอบผลการทดลองของแต่ละ Bug ได้ตั้งแต่ขั้นตอนการ Compile การ Execute บน Buggy Version และ Fixed Version การพิจารณา Fault Detection ไปจนถึงการวัด Test Coverage นอกจากนี้ การจัดเก็บ Prompt, Response, Test Code และผลการประเมินแยกตาม Bug ยังช่วยสนับสนุนการตรวจสอบย้อนกลับและการทำซ้ำการทดลองภายใต้ Configuration เดียวกันได้อย่างเป็นระบบ

บทที่ 7

สรุปผลและข้อเสนอแนะ

7.1 สรุปผลการศึกษา

โครงการนี้ได้ศึกษาแนวทางการสร้างชุดทดสอบอัตโนมัติจำนวน 4 แนวทาง โดยแบ่งออกเป็น Automated Test Case Generation Algorithms จำนวน 2 แนวทาง ได้แก่ Whole Test Suite Generation (WTS) และ Coverage-guided Greybox Fuzzing และ AI-Assisting Tools จำนวน 2 แนวทาง ได้แก่ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash โดยใช้ Defects4J เป็นชุดข้อมูลสำหรับประเมินความสามารถของชุดทดสอบในการตรวจจับข้อบกพร่องจริงของโครงการภาษา Java

ในส่วนของ Automated Test Case Generation Algorithms ได้ศึกษา Whole Test Suite Generation ผ่านเครื่องมือ EvoSuite ซึ่งใช้แนวคิด Search-Based Software Testing ในการค้นหาและปรับปรุง Test Suite โดยอาศัย Fitness และ Coverage เป็นข้อมูลสำหรับประเมินคุณภาพของชุดทดสอบ นอกจากนี้ยังได้ศึกษา Coverage-guided Greybox Fuzzing ซึ่งใช้ Coverage Feedback เพื่อควบคุมกระบวนการสร้างและปรับเปลี่ยน Input เพื่อค้นหาเส้นทางการทำงานใหม่ของโปรแกรม

ในส่วนของ AI-Assisting Tools ได้ศึกษา Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ผ่าน KIU IntelSphere API โดยใช้ความสามารถของ Generative AI ในการวิเคราะห์ข้อมูลของ Bug, Trigger Test และ Source Code Diff แล้วสร้าง Test Code ระดับ Unit Testing จาก Prompt ที่สร้างขึ้นตามข้อมูลของแต่ละ Bug จากนั้นนำ Test ที่สร้างขึ้นไปประเมินกับ Buggy Version และ Fixed Version เพื่อพิจารณาความสามารถในการตรวจจับข้อบกพร่อง

จากผลการทดลองของ Gemini 3.8 Flash พบว่าสามารถตรวจจับข้อบกพร่องได้ 106 จาก 131 Bug ที่สามารถวัดผลได้ คิดเป็น Fault Detection Rate (FDR) เท่ากับ 80.92% โดย Collections มีค่า FDR สูงที่สุดเท่ากับ 92.86% ขณะที่ JacksonCore มีค่า FDR ต่ำที่สุดเท่ากับ 57.69% ส่วนค่าเฉลี่ย Line Coverage ของโครงการที่นำมาประเมินอยู่ในช่วง 19.9%–49.2%

อย่างไรก็ตาม ผลดังกล่าวยังไม่สามารถใช้สรุปว่า Gemini 3.8 Flash มีประสิทธิภาพเหนือกว่า Whole Test Suite Generation หรือ Coverage-guided Greybox Fuzzing เนื่องจากแต่ละแนวทางมีจำนวน Bug ชุดข้อมูล และเงื่อนไขการทดลองแตกต่างกัน อีกทั้งผลการทดลองของ WTS และ Fuzzing ยังไม่ครอบคลุมข้อมูลสำหรับการเปรียบเทียบเชิงตัวเลขอย่างสมบูรณ์

ดังนั้น ผลการศึกษาที่น่าเสนอในโครงการนี้ควรพิจารณาตามขอบเขตและเงื่อนไขของข้อมูลที่ได้จากการทดลอง โดยการเปรียบเทียบประสิทธิภาพของทั้ง 4 แนวทางอย่างสมบูรณ์ควรดำเนินการเมื่อมีข้อมูลจากทุกแนวทางภายใต้เงื่อนไขที่สามารถเปรียบเทียบกันได้

7.2 สรุปผลการเปรียบเทียบทั้ง 4 แนวทาง

ผลการศึกษาพบว่า การสร้างชุดทดสอบอัตโนมัติสามารถดำเนินการได้หลายแนวทาง โดยแต่ละแนวทางมีหลักการทำงานและจุดเด่นแตกต่างกัน Whole Test Suite Generation ใช้ Search-Based Software Testing เพื่อค้นหาและปรับปรุง Test Suite ตามเป้าหมายด้าน Fitness และ Coverage ขณะที่ Coverage-guided Greybox Fuzzing ใช้การสร้างและปรับเปลี่ยน Input ร่วมกับ Coverage Feedback เพื่อสำรวจเส้นทางการทำงานของโปรแกรม

สำหรับ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ซึ่งเป็น AI-Assisting Tools สามารถใช้ความสามารถของ Generative AI ในการวิเคราะห์ Source Code และข้อมูลบริบทของ Bug เพื่อสร้าง Test Code จาก Prompt โดยไม่จำเป็นต้องกำหนด Test Case แต่ละรายการด้วยตนเอง

เมื่อพิจารณาลักษณะของกระบวนการพบว่า WTS และ Coverage-guided Greybox Fuzzing มีลักษณะสำคัญในการใช้ข้อมูลจากการ Execute Program เป็น Feedback สำหรับการค้นหา Test หรือ Input ใหม่ ขณะที่ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ใช้ความสามารถในการวิเคราะห์ Source Code และข้อมูลบริบทเพื่อสร้าง Test Code โดยตรง

แต่ละแนวทางจึงมีข้อดีและข้อจำกัดแตกต่างกัน WTS เหมาะกับการค้นหา Test Suite ตามเป้าหมายที่กำหนด Coverage-guided Greybox Fuzzing มีจุดเด่นในการสำรวจ Input Space และค้นหา Edge Case ส่วน AI-Assisting Tools มีความสามารถในการสร้าง Test จาก Source Code และข้อมูลของ Bug อย่างไรก็ตาม Test ที่สร้างโดย AI อาจพบปัญหาด้านการ Compile, Assertion, Dependency และความสมบูรณ์ของ Response

จากข้อมูลที่ได้จากการทดลองจึงยังไม่สามารถระบุได้ว่าแนวทางใดมีประสิทธิภาพสูงที่สุด เนื่องจาก WTS และ Coverage-guided Greybox Fuzzing ยังมีข้อมูลผลการทดลองไม่ครบถ้วน และ Claude AI Sonnet 5.5 กับ Gemini 3.8 Flash ยังไม่ได้ทดลองกับชุด Project และ Bug เดียวกันทั้งหมด

7.3 ข้อจำกัดที่มีผลต่อการสรุปผล

ข้อจำกัดด้าน API Quota เป็นปัจจัยสำคัญที่ส่งผลต่อจำนวน Bug และจำนวนโครงการที่สามารถนำมาทดลองด้วย AI-Assisting Tools ทำให้การทดลองยังไม่ครอบคลุมทุกโครงการใน Defects4J ตามขอบเขตที่กำหนดไว้

นอกจากนี้ ผลการทดลองของ Compress ยังไม่ครอบคลุม Bug ทั้งหมด เนื่องจาก Bug 46 และ Bug 47 ยังไม่ได้ดำเนินการ ขณะที่การทดลอง Gemini 3.8 Flash ยังไม่ครอบคลุมโครงการ Chart, Lang, Jsoup, Math, JacksonDatabind และ Closure ความแตกต่างของชุด Project และ Bug ที่ใช้กับ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash รวมถึงความแตกต่างของ Prompt และเงื่อนไขในการทดลอง ยังเป็นปัจจัยที่ต้องพิจารณาเมื่อวิเคราะห์ผล Fault

Detection Rate ดังนั้นค่า FDR ของทั้งสองเครื่องมือจึงไม่ควรถูกนำมาใช้เป็นหลักฐานเพียงอย่างเดียวในการสรุปประสิทธิภาพของโมเดล

ด้วยข้อจำกัดดังกล่าว ผลการทดลองที่ได้จึงยังไม่สามารถใช้เป็นตัวแทนของประสิทธิภาพของ AI-Assisting Tools บนชุดข้อมูล Defects4J ทั้งหมด และการเปรียบเทียบเชิงตัวเลขระหว่างทั้ง 4 แนวทางควรดำเนินการภายใต้ชุดข้อมูลและเงื่อนไขการทดลองที่สามารถเปรียบเทียบกันได้

7.4 ข้อเสนอแนะสำหรับการดำเนินงานต่อไป

1. ดำเนินการทดลอง Whole Test Suite Generation ให้ครอบคลุมชุด Bug ที่กำหนด และบันทึกค่า Branch Coverage, Line Coverage, Mutation Coverage, Fault Detection Rate และระยะเวลาในการสร้างชุดทดสอบ
2. ดำเนินการทดลอง Coverage-guided Greybox Fuzzing ตาม Configuration ที่กำหนด โดยใช้ Budget 30, 60 และ 120 วินาที และดำเนินการซ้ำ 5 ครั้งต่อ Configuration เพื่อคำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐาน
3. ขยายการทดลอง AI-Assisting Tools ให้ครอบคลุมโครงการและ Bug ที่เหลือใน Defects4J เพื่อเพิ่มความครอบคลุมของข้อมูลสำหรับการวิเคราะห์ผล
4. ดำเนินการทดลอง Gemini 3.8 Flash ให้ครบ Compress Bug 46 และ Bug 47 เพื่อให้ผลการทดลองของโครงการ Compress มีความสมบูรณ์มากขึ้น
5. ทดลอง Claude AI Sonnet 5.5 และ Gemini 3.8 Flash กับชุด Project และ Bug เดียวกัน เพื่อให้สามารถเปรียบเทียบความสามารถในการตรวจจับข้อบกพร่องได้โดยตรง
6. ควบคุมโครงสร้างและเนื้อหาของ Prompt ของ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ให้มีเงื่อนไขใกล้เคียงกันมากขึ้น เพื่อควบคุมปัจจัยที่อาจส่งผลต่อความแตกต่างของผลการทดลอง
7. ตรวจสอบ Test Code ที่ Compile ไม่ผ่านเป็นรายกรณี เพื่อแยกข้อผิดพลาดที่เกิดจาก Test Code ออกจากปัญหาที่เกิดจาก Helper, Classpath หรือ Environment
8. ตรวจสอบและเปรียบเทียบผลจาก Coverage Script รุ่นเดิมและรุ่นที่ได้รับการปรับปรุง เพื่อยืนยันความสอดคล้องของกระบวนการวัดผลก่อนนำข้อมูลจากทั้งสองรุ่นมารวมกัน
9. จัดทำตารางเปรียบเทียบผลการทดลองของทั้ง 4 แนวทางเมื่อมีข้อมูลครบถ้วน โดยพิจารณา Fault Detection Rate, Test Coverage, จำนวน Test, ระยะเวลาในการสร้างชุดทดสอบ และความสามารถในการ Compile และ Execute
10. วิเคราะห์ความสัมพันธ์ระหว่าง Test Coverage และ Fault Detection Rate เพื่อประเมินคุณภาพของชุดทดสอบอย่างรอบด้าน และหลีกเลี่ยงการใช้ Coverage เพียงตัวเดียวในการสรุปประสิทธิภาพของแต่ละแนวทาง

เอกสารอ้างอิง

1. Fraser, G., & Arcuri, A. (2011). *Evolutionary Generation of Whole Test Suites*. International Conference on Quality Software.
2. Arcuri, A., & Fraser, G. (2014). *On the Effectiveness of Whole Test Suite Generation*. International Symposium on Search Based Software Engineering.
3. Fraser, G., & Arcuri, A. (2025). *A Retrospective on Whole Test Suite Generation: On the Role of SBST in the Age of LLMs*. IEEE Transactions on Software Engineering.
4. Just, R., Jalali, D., & Ernst, M. D. (2014). *Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs*. Proceedings of the International Symposium on Software Testing and Analysis.
5. Zalewski, M. *American Fuzzy Lop (AFL)*.
6. Padhye, R., et al. (2019). *JQF/Zest: Coverage-Guided, Semantic Fuzzing for Java*. International Symposium on Software Testing and Analysis.
7. Gay, G., Staats, M., Whalen, M., & Heimdahl, M. (2015). *The Risks of Coverage-Directed Test Case Generation*. IEEE Transactions on Software Engineering.
8. EvoSuite. *Automated Test Generation for Java*.
9. Defects4J. *A Database of Real Faults in Java Programs*.